

CSE450 Capstone Project Report

Exploratory Testing Automation

Submitted by

Khalid Hasan Tuhin	2105002
Mehemud Azad	2105014
Niloy Kumar Mondal	2105044
Mohammad Raihan Rashid	2105046
Jonayed Mohiuddin Shanto	2105060

Supervised by

Dr. A. B. M. Alim Al Islam
Professor, Department of CSE, BUET



Department of Computer Science and Engineering
Bangladesh University of Engineering and Technology
Dhaka, Bangladesh

Session 2024–2025
September 2026

CANDIDATES' DECLARATION

We declare that the work presented in this report, titled “Exploratory Testing Automation”, is the outcome of the engineering work carried out by us under the supervision named on the title page.

All prior work, third-party code, libraries, data, copyrighted material, and tools used in the project have been properly acknowledged. Any use of generative AI tools has been disclosed. We accept accountability and responsibility for the accuracy and integrity of this report and for our individual contributions to the project.

Khalid Hasan Tuhin (2105002)

Mehemud Azad (2105014)

Niloy Kumar Mondal (2105044)

Mohammad Raihan Rashid (2105046)

Jonayed Mohiuddin Shanto (2105060)

ACKNOWLEDGEMENT

Thank your supervisor, and anyone else who helped: domain experts, collaborators, users who tested the system, people from other departments or organisations you consulted.

ABSTRACT

An automated tool can tell you that an application crashed. It cannot tell you that the application quietly charged the wrong price, because it has no idea what the right price was. That gap, between detecting a failure and detecting a *wrong answer*, is why graphical user interface testing is still done by people, at a cost that grows with every release, and whose hard-won knowledge of an application’s weak points leaves with the tester.

Samsung Research and Development Institute Bangladesh posed the problem and asked for a demanding change: to move from *directed* testing, where an agent is given an objective, to *exploratory* testing, where the agent decides what is worth testing next, infers what correct behaviour would be, and is better informed next time. The objective stops being an input and becomes an output, and a failure becomes ambiguous between a real defect, a limitation of the agent, and an environment that simply blocked it.

The system built in response is an autonomous exploratory testing agent organised around one persistent knowledge graph. Three stateless language-model roles (a planner that writes the next test case, an executor that drives the application, and an investigator that distils each run into findings) communicate only through that graph. It grows from a requirements specification, a design export and a defect history, and from the agent’s own observations, so it works where no documentation exists. One planner and one graph drive both Android and the web. All eight of the partner’s requirement families were delivered, plus ten derived on contact with a real application. Among them are a fault attribution taxonomy that stops the agent’s own failures being reported as defects, and a safety boundary around irreversible actions.

Validation combined a regression suite, live campaigns against a production commerce application and a website, and a reading of seven research systems. The agent models an application it has never seen, cites the requirement each test covers, and costs about one US cent per test, orders of magnitude below published figures. Autonomy and requirement coverage remain modest and are reported unadjusted. The review also yielded a finding that constrains every claim in this field, this project’s included: none of the seven systems reports precision or recall for defect detection, and the two that do injected their faults by hand, because without a specification there is no oracle beyond a crash, and without an oracle no ground truth. Grounding the agent in a specification attacks that problem directly; the seeded-defect experiment that would close it is the most valuable work remaining.

Keywords: exploratory testing; GUI test automation; LLM agents; knowledge graphs.

Contents

Candidates' Declaration	i
Acknowledgement	ii
Abstract	iii
1 Problem Identification and Formulation	1
1.1 Background and Motivation	1
1.1.1 The Shift the Industry Partner Asked For	2
1.1.2 Why Exploratory Testing Is Hard	2
1.1.3 The Gap	3
1.2 Problem Statement	4
1.3 Complexity of the Problem	6
1.4 Project Objectives and Scope	9
2 Preliminary Research	12
2.1 Literature Review	12
2.1.1 From Random Input to Learned Exploration	12
2.1.2 The Language-Model Agent Wave	13
2.1.3 What the Evidence Agrees On	14
2.1.4 How This Literature Is Measured, and Why It Reports Almost No Precision or Recall	15
2.1.5 What the Literature Leaves Open	17
2.1.6 Additional Works Surveyed	18
2.2 Existing Solutions and Technologies	20
2.3 Market and User Research	22

2.4	Feasibility Study	23
3	Requirements Analysis	26
3.1	Project Objectives and Constraints	26
3.1.1	The Primary Requirement: From Directed to Exploratory Testing . . .	26
3.1.2	Stakeholders and Their Expectations	27
3.1.3	Functional Requirements	28
3.1.4	Non-Functional Requirements	33
3.1.5	Constraints	33
3.2	Regulatory Requirements and Standards	34
3.3	Formulation of Solutions	37
4	System Design and Implementation	39
4.1	Functional Design and Sub-problem Partitioning	39
4.2	Design Refinement and Simulation	39
4.3	Application of Tools	39
4.4	Implementation	39
4.5	Deployment	40
5	Investigation and Evaluation	41
5.1	Experimental Investigation	41
5.2	Performance Evaluation	41
5.3	Deviations and Design Revisions	41
6	Project Management, Teamwork, and Impact	42
6.1	Project Management and Finance	42
6.1.1	Planning and Risk Management	42
6.1.2	Budgeting and Resource Identification	42
6.1.3	Economic Analysis and Financial Prospecting	42
6.1.4	Sustainability in Business and Commercialisation	43

6.2	Individual and Teamwork	43
6.2.1	Participation and Contribution	43
6.2.2	Collaboration and Conflict Resolution	43
6.2.3	Leadership and Direction	43
6.2.4	Multidisciplinary Engagement	43
6.3	Impact on Society	43
6.4	Environmental Impact and Life Cycle Sustainability	46
6.5	Ethics	48
6.5.1	Equity and Inclusivity	48
6.5.2	Accountability and Personal Responsibility	49
6.5.3	Proper Use of Intellectual Property	49
6.5.4	Professionalism and Ethical Codes	50
7	Conclusion	52
7.1	Conclusion and Future Work	52
7.2	Continuous Engagement and Self-directed Learning	58
	References	61

List of Figures

4.1	System architecture	39
6.1	Project schedule	42

List of Tables

1.1	Complex engineering problem characteristics (P1–P7) and evidence from this project	6
1.2	Knowledge profiles referenced by P1	10
2.1	The seven primary systems reviewed: goal, core mechanism, and the basis on which each was evaluated	13
2.2	The three evaluation regimes in the reviewed literature, and the limit of each . .	16
2.3	Deployable technologies in the automated GUI testing space	20
3.1	Stakeholders, expectations and conflicts	27
3.2	Partner requirements (ETA-TR-2026-001) and their realisation	29
3.3	Requirements derived by the team, beyond ETA-TR-2026-001	30
3.4	Standards and regulatory instruments, applicability and compliance	35
3.5	Evaluation of alternative solutions against the project’s criteria. Legend: ✓ meets, ~ partially meets, × does not meet.	38
7.1	Outcome against each stated objective	52

Chapter 1

Problem Identification and Formulation

1.1 Background and Motivation

Graphical user interfaces are the surface through which almost all consumer software is used, and they are the surface on which almost all of its defects become visible to a user. Verifying them is therefore unavoidable, and it is expensive. A modern mobile application ships on a fortnightly or monthly cadence, across several device profiles (phone, tablet, watch, television) and several platform versions, and every release re-opens the question of whether the parts that used to work still work.

Industry answers this question in three broad ways, and each of them is unsatisfactory in a different direction.

Manual exploratory testing is what skilled human testers do: they open the application, form a hypothesis about where it is likely to be weak, probe it, and follow whatever they find. It is by a wide margin the most effective technique at finding defects that nobody anticipated, because the tester is reasoning about the product rather than replaying a script. Its weaknesses are economic and organisational rather than technical. It does not scale with the number of device–platform–profile combinations, the cost is linear in the number of releases, and, most damagingly, the knowledge the tester builds up about where the application is fragile lives in that person’s head. When they move to another project, the organisation starts again.

Scripted automation (Appium, Espresso, Selenium, and the page-object patterns built on them) converts a tester’s knowledge into code, which makes it repeatable and cheap to re-run. The cost moves from execution to maintenance. Every script encodes a set of assumptions about element identifiers, screen layout, and navigation order, and a user-interface redesign invalidates all of them at once. More fundamentally, a script can only check what its author thought to check: scripted automation is a regression technique, not a discovery technique. It confirms known behaviour; it does not explore.

Random and search-based exploration (Android’s Monkey [1], and the academic line of work on search-based GUI exploration) removes the human from the loop entirely and drives the application with generated input. It is cheap and it scales, but it has no test oracle worth the

name. It can detect a crash or an unhandled exception, because those are self-evident failures, but it cannot detect that an application accepted an invalid price, routed an order to the wrong seller, or displayed a total that does not match its line items. Those are the defects that matter commercially, and they are invisible without a notion of what the application was supposed to do.

1.1.1 The Shift the Industry Partner Asked For

The problem addressed here was posed by an industry partner, Samsung Research and Development Institute Bangladesh, whose brief is what fixes the project's difficulty. The partner supplied the problem, a technical requirements document and an early review; the design, implementation and evaluation reported in this document are the team's own work. That review asked for one specific change: to move from *directed functional testing*, in which an agent is handed a test objective and must carry it out, to *exploratory testing*, in which the agent must decide for itself what is worth testing next. The technical requirements document that followed, ETA-TR-2026-001 [2], formalises that shift as the evolution of a static-knowledge test generator into "an autonomously learning, self-optimising exploratory test engineer", and states the objective in one sentence that this project is ultimately evaluated against: *every execution cycle must make the agent smarter than the last*.

The distinction is easy to state and easy to underestimate. In directed testing the objective is an *input*: a person, or an upstream tool, has already decided what to check and usually what the correct answer is, and the agent's job is execution. In exploratory testing the objective is an *output*: the agent must generate the objective, derive its own notion of correct behaviour, execute, judge the result, and then let what it learned change the next objective it generates. Exploratory testing is characterised by Kaner, Bach and Pettichord as the *simultaneous* performance of learning, test design and test execution [?]; automating it means automating all three at once, and automating the feedback between them.

1.1.2 Why Exploratory Testing Is Hard

Six difficulties follow from that definition, and together they account for most of the engineering effort reported in this document.

- D1. The objective must be generated, and the search space is unbounded.** For any non-trivial application the set of possible action sequences is effectively infinite, and there is no ground truth for which of them is worth performing. A directed agent searches for a path to a given goal; an exploratory agent must first search the space of goals.
- D2. There is no stopping criterion.** A scripted suite is finished when its last case has run. Exploration ends when the budget ends, so every reported result is conditioned on a

budget and no result is ever “complete”. Coverage becomes something that is reported rather than something that is achieved.

- D3. The oracle must be derived rather than supplied.** A directed test case arrives with its expected result attached. An exploratory agent must work out what correct behaviour is from whatever specification exists and, critically, must abstain when the specification is silent rather than inventing an expectation. The alternative failure modes are a fabricated defect report and a silent pass, and Section 2.1.3 shows both are documented in the literature.
- D4. Credit assignment is ambiguous.** When a scripted test fails, the script is fixed and the application is the variable. When an exploratory run ends without a pass, three explanations compete: the application is wrong, the agent could not drive the application, or the environment blocked the attempt. Only the first is a defect, and conflating them inflates the reported defect count, which is the most expensive error this kind of system can make.
- D5. Non-reproducibility is intrinsic, not incidental.** Two exploratory runs are *supposed* to do different things; that is where the value comes from. A regression therefore cannot be demonstrated by re-running the suite, and every claim about the system must be made over a distribution of runs rather than a single run.
- D6. Without memory, exploration is worthless.** A stateless exploratory agent re-treads the same ground every session. The whole value of the technique depends on what it learned persisting and changing what it does next, which is precisely why the partner’s requirements are dominated by mechanisms for *remembering* (Section 3.1.3).

The literature confirms that this is the harder problem rather than merely a different one. Of the seven agent systems reviewed in Chapter 2, five are handed their objective: a task string, a test case, or a natural-language test requirement [3–7]. Only DroidAgent generates its own testing goals [8], and its authors describe it as the first Android GUI testing technique able to do so. The measured gap is instructive: objective-driven systems report 77% to 88.55% task completion [6, 7], while DroidAgent completes 59% of the tasks it set itself, with a further 15% judged not viable in the first place. Those figures come from different applications and benchmarks and are not a controlled comparison, but the direction is consistent and unsurprising: an agent that must invent its own objective will set some objectives that cannot be achieved.

1.1.3 The Gap

The gap this project addresses is therefore specific: an automated technique that has the *oracle* of manual exploratory testing, meaning a model of intended behaviour drawn from the specification

and from what has gone wrong before, together with the *scale and repeatability* of automation, and that, unlike both, *accumulates* what it learns so that the hundredth execution is better informed than the first.

The stakeholders affected are the partner's quality assurance engineers, whose manual effort the system is intended to displace; application developers, who receive its defect reports and pay the cost of false positives; and, indirectly, end users of the applications under test.

1.2 Problem Statement

Statement

Given (i) whatever partial specification of an application exists (a software requirements specification, a user-interface design export, a history of previously filed defects, or none of these) and (ii) an executable instance of that application on a real device or in a browser, construct an autonomous agent that *chooses* what to test next, executes its choice, judges the outcome against a derived oracle, and persists what it learned, such that the knowledge produced by each execution measurably improves the test cases proposed in subsequent executions. No test objective is supplied from outside the loop.

Engineering formulation

Let A be the application under test and let K_t be the agent's knowledge state at round t . The system is a loop

$$K_{t+1} = U(K_t, E(A, P(K_t))),$$

where P is a *planner* that maps a knowledge state to a concrete test case, E is an *executor* that drives A through that test case and returns an observed trajectory, and U is an *investigator* that reduces a trajectory to durable, deduplicated knowledge and merges it into the graph. The three functions are realised by separate large-language-model roles and communicate only through K , which is a persistent property graph. No conversational state is carried between calls.

The exploratory character of the problem is carried entirely by P . A directed testing system would take the test case as an input and need only E ; here P must select from an unbounded space using K_t as its only evidence, and the quality of the whole system is bounded by the quality of that selection.

Inputs

A requirements specification (free text), a user-interface design export, a structured defect

history, and the live accessibility tree plus screenshot of the running application. Every one of these is optional; the system must degrade rather than fail when a source is absent.

Outputs

Executable test cases with steps, expected results and requirement citations; per-execution verdicts with an explicit fault attribution; atomic, deduplicated findings about the application; and a coverage and risk report over the requirement set.

Entities

Requirements, requirement chunks, feature areas, design screens and elements, observed UI states and the transitions between them, navigation paths, defects, test cases, execution logs, error patterns, strategy memory, and findings, all of them nodes in one project-scoped property graph.

Quantities to be optimised

Requirement coverage per unit cost; autonomy, defined as the fraction of executions that terminate with a verdict attributable to the application rather than to the agent or the environment; defect-discovery rate; duplicate-proposal rate (to be minimised); and monetary cost per executed test.

Quantities to be guaranteed

No destructive or irreversible action on the application under test; no silent failure, meaning every degradation must be recorded and surfaced; and no fabricated evidence, meaning a verdict must be traceable to an observation.

Assumptions

The application exposes an accessibility tree (Android) or a DOM (web); a dedicated, disposable test account is available; the device or browser is reachable from the host; and a hosted language-model API is available within a fixed budget.

Engineering knowledge demanded

The formulation above is not solvable by routine application of a single body of knowledge. It requires, at minimum: graph data modelling and query design (the knowledge state is a property graph with constraints and vector indexes); information retrieval, including both lexical and dense-vector similarity, to select the fraction of K_i that fits in a bounded prompt; similarity and hashing theory, applied to the non-trivial question of when two observed screens are the same screen; distributed-systems design, since the four processes must share state without sharing memory; the specialist body of knowledge of software testing, in particular test-oracle theory and fault attribution; and empirical evaluation methodology, because the central claim of the project, that the agent improves, is a claim about a trend and can only be established statistically.

1.3 Complexity of the Problem

A complex engineering problem must have characteristic P1 and some or all of P2–P7 (Table 1.1). P1 requires in-depth knowledge at the level of one or more of the knowledge profiles K3–K8 listed in Table 1.2.

Table 1.1: Complex engineering problem characteristics (P1–P7) and evidence from this project

Code	Definition	Evidence from this project
P1	Depth of knowledge required. Cannot be resolved without in-depth engineering knowledge at the level of one or more of K3, K4, K5, K6, or K8, which allows a fundamentals-based, first-principles analytical approach.	K3, K4, K5, K6 and K8 are all engaged. K3: the state-identity problem was solved from first principles as a cascade of set-similarity measures (exact signature, Jaccard index, containment, structural-skeleton containment) with a perceptual image hash as the final tie-break, each with an analytically chosen threshold and guard. K4: test-oracle theory and fault attribution, taken from the software-testing body of knowledge, are what separate an application defect from an agent failure. K5: the four-process architecture and the single-writer graph discipline are design-knowledge decisions. K6: current practice in Android accessibility services, Playwright, Neo4j and hosted language-model tool-calling. K8: seven recent systems were read in full and a further seven surveyed, as reported in Chapter 2.

continued on the next page

Table 1.1: Complex engineering problem characteristics (P1–P7) (*continued*)

Code	Definition	Evidence from this project
P2	Range of conflicting requirements. Involves wide-ranging or conflicting technical, engineering, and other issues.	Four irreducible conflicts were managed rather than eliminated: <i>autonomy versus safety</i> , where the agent must act freely but must never log out, delete an account, or make a payment; <i>coverage versus cost</i> , where every additional test is a metered charge against a fixed budget; <i>generality versus grounding</i> , where knowledge of the specific application makes the agent effective and simultaneously makes it useless on the next application; and <i>exploration versus reproducibility</i> , which is difficulty D5, where the agent's value comes from non-deterministic exploration while the report it produces must be defensible evidence.
P3	Depth of analysis required. Has no obvious solution and requires abstract thinking and originality in analysis to formulate suitable models.	Three sub-problems had no off-the-shelf answer. (i) <i>What should be tested next?</i> This is difficulty D1, and it has no ground truth; the answer adopted is to rank an unbounded space by evidence held in the knowledge graph (uncovered requirements, defect-dense areas, regression risk) rather than by any fixed heuristic. (ii) <i>When are two screens the same screen?</i> A naive signature fragmented one wizard into nine distinct states, destroying the navigation model; the multi-stage cascade described under P1 was designed in response. (iii) <i>Whose fault was this failure?</i> Difficulty D4: an execution that ends without a pass is ambiguous between a defect, an agent limitation and an environmental block, and a three-way attribution taxonomy with a single authoritative definition was introduced to prevent conflation.

continued on the next page

Table 1.1: Complex engineering problem characteristics (P1–P7) (*continued*)

Code	Definition	Evidence from this project
P4	Familiarity of issues. Involves infrequently encountered issues.	The system embeds non-deterministic components inside a harness whose own output is a correctness judgement, an inversion of the usual assumption that a test harness is deterministic. Further non-routine issues: applications built with cross-platform toolkits expose no stable resource identifiers and report a single activity on every screen, defeating conventional element location; and prompt injection becomes a live concern because the application's own on-screen text is fed to a model that also decides what to do next.
P5	Extent of applicable codes. Is outside problems encompassed by standards and codes of practice for professional engineering.	ISO/IEC/IEEE 29119 [9] gives a vocabulary and a process for testing but presumes that test cases are authored by people and that test design is deterministic. It has nothing to say about an agent that authors its own test cases at run time, nor about how to report coverage when the test set is not fixed in advance, which is exactly the regime difficulty D2 describes. No standard governs the correctness of a language-model-produced verdict. The applicable codes therefore had to be supplemented by project-specific rules, documented in Section 3.2.

continued on the next page

Table 1.1: Complex engineering problem characteristics (P1–P7) (*continued*)

Code	Definition	Evidence from this project
P6	Extent of stakeholder involvement and conflicting requirements. Involves diverse groups of stakeholders with widely varying needs.	The industry partner wants learning that transfers across device profiles and platforms; a quality-assurance lead wants runs that can be reproduced and audited; application developers want a near-zero false-positive rate, because each false defect report costs them investigation time; the academic supervisor requires a defensible empirical evaluation; and the end users of the application under test require that testing never damage live data. These pull in different directions, most sharply transfer-driven generalisation against reproducibility.
P7	Interdependence. Is a high-level problem including many component parts or sub-problems.	The partner's requirements document itself decomposes the work into eight requirement families (ETA-REQ-301 to 308) with an explicit dependency ordering, and the delivered system comprises four cooperating processes, two execution platforms (Android and web), three distinct language-model roles, nine pluggable knowledge sources and a single shared graph schema. A change to the attribution taxonomy, for example, propagates to the executor, the investigator, the risk score, the coverage report and the generated PDF.

1.4 Project Objectives and Scope

Objectives

The objectives below are stated so that each one can be checked; Chapter 5 evaluates the delivered system against them. O2 and O4 together are the exploratory requirement of Section 1.1.1: the agent must choose its own objectives, and what it learns must change the choices it makes

Table 1.2: Knowledge profiles referenced by P1

Code	Description
K3	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.
K4	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.
K5	Knowledge that supports engineering design in a practice area.
K6	Knowledge of engineering practice (technology) in the practice areas in the engineering discipline.
K8	Engagement with selected knowledge in the research literature of the discipline.

afterwards.

- O1. Build a persistent, project-scoped knowledge graph** that ingests a requirements specification, a user-interface design export and a defect history, and that is additionally grown from the agent’s own observations of the running application. *Verified by:* ingesting each source independently and querying the resulting nodes and relationships.
- O2. Select and generate executable, non-duplicate test cases from that graph without an externally supplied objective**, each citing the requirement it covers. *Verified by:* the proportion of generated test cases that carry a valid requirement citation, the rejection rate of re-worded duplicates, and the absence of any human-supplied objective in the loop.
- O3. Execute generated test cases autonomously** against a real Android device and against a live web application, and return a verdict with an explicit three-way fault attribution. *Verified by:* the autonomy rate, meaning executions that terminate with a verdict about the application rather than about the agent, measured over a campaign.
- O4. Close the learning loop**, so that execution outcomes are persisted as durable findings, navigation paths, error patterns and strategy scores, and demonstrably alter the next round’s proposals. *Verified by:* showing that a proposal changes when, and only when, the corresponding knowledge is present in the graph.
- O5. Satisfy the eight requirement families of ETA-TR-2026-001 [2]** (ETA-REQ-301 to 308) and its two non-functional requirements. *Verified by:* the requirement-to-implementation traceability matrix in Section 3.1, and the partner’s stated acceptance criteria.
- O6. Keep the system application-agnostic.** No knowledge of any particular application may be embedded in code or prompts; all application-specific information must arrive through configuration. *Verified by:* an automated generality audit over the source tree, and by running the same unmodified system against more than one application.

- O7. Operate within a fixed monetary budget**, with the cost per executed test measured and reported rather than estimated. *Verified by:* reconciling the provider's billing record against the number of executions.
- O8. Provide an operator surface**: a live dashboard and an exportable report, so that a run can be watched, audited and handed to someone who was not present for it. *Verified by:* demonstration against a live campaign.

Scope

Within scope. Autonomous exploratory testing of (i) Android applications on a physical device or emulator, driven through the platform accessibility service, and (ii) web applications, driven through a headless or headed browser; ingestion of requirements, design exports and defect histories; a persistent knowledge graph with learning across runs; fault attribution and self-healing retry; risk-weighted prioritisation; and an operator dashboard with report export.

Explicitly out of scope. Unit, integration, performance, load and security testing of the application under test; this project addresses system-level testing through the user interface only. Directed execution of a human-authored test suite: the system may be given a specification, never an individual objective, since supplying objectives would remove the exploratory problem the partner asked to have solved. Testing of flows that cannot be completed without an irreversible side effect, specifically account registration, one-time-password verification, payment and account deletion; these are guarded against rather than exercised, and the corresponding requirements are marked as preconditions already satisfied. Automated repair of discovered defects. Training or fine-tuning of language models: the project uses hosted models through an API and treats them as components. Formal certification of the agent's verdicts; a reported defect is a candidate for human confirmation, not a confirmed defect.

Deferred. Cross-device-profile transfer beyond the application axis, and the seeded-defect experiment needed to measure precision and recall against a known ground truth, are specified and partially implemented but not validated within this session; both are carried into Chapter 7 as future work.

Chapter 2

Preliminary Research

2.1 Literature Review

Seven primary systems were read in full and form the evidentiary basis of this review: four are GUI *testing* systems [6–8, 10] and three are GUI *task-automation* agents [3–5] whose memory and grounding mechanisms transfer directly to the testing problem. Every figure quoted below was taken from those papers. The review is organised around four questions: how the field arrived at language-model agents, what those agents demonstrably add, how the resulting claims are measured, and what the literature leaves open.

2.1.1 From Random Input to Learned Exploration

Automated Android GUI testing has passed through three generations. The first drives the application with pseudo-random events; Android’s own Monkey is the canonical example and remains the universal baseline [1]. The second is systematic or model-based: DroidBot builds a UI transition graph and greedily prioritises unexplored widgets [11]. The third is learning-based: Humanoid trains a deep network on real human interaction traces to bias action selection toward human-like paths [12], and a parallel line applies reinforcement learning with curiosity rewards to reach novel states.

Liu et al. summarise the limitations of the learning-based generation precisely [10]: these methods require large interaction datasets that are difficult to collect; they generalise poorly to applications they were not trained on, while applications evolve continuously; and the environment is non-deterministic. Their own example is that deleting the last item from a list produces an empty list on which the delete control no longer works, which defeats a learned policy.

A second, more fundamental criticism runs through the whole lineage and is the one this project takes as its starting point. All three generations optimise *structural* objectives: activity coverage, widget coverage, code coverage. Yoon et al. point out that this is not what practitioners want [8]: an empirical study of Android developers found that the dominant test-design strategy is to follow the usage model of the application, that developers prefer test cases targeting individual features and use cases, and that, asked what form they would like generated tests to take, they

chose natural language over test-API scripts, with expected outputs and use-case steps included. Coverage is a proxy that the people who consume the results did not ask for [13].

2.1.2 The Language-Model Agent Wave

Between 2023 and 2025 the field reorganised around large-language-model agents. Table 2.1 summarises the seven systems reviewed.

Table 2.1: The seven primary systems reviewed: goal, core mechanism, and the basis on which each was evaluated

System	Goal and mechanism	Evaluation basis and headline result
GPTDroid [10]	GUI <i>testing</i> framed as a question-and-answer task; functionality-aware memory prompting carries testing knowledge across the whole session.	93 Android applications carrying 143 previously confirmed crash bugs. 75% activity and 66% code coverage against 57% for the best baseline; 95 of the known bugs detected, 31% more than the best baseline, and 35% faster. Separately, 53 previously unknown crashes on Google Play, of which 35 were confirmed or fixed.
DroidAgent [8]	GUI <i>testing</i> with four cooperating agents (Planner, Actor, Observer, Reflector) over short-term, long-term and spatial memory; sets its own natural-language task goals.	15 applications from the Themis benchmark [14], two-hour budget each. 61% activity coverage against 51% for Humanoid; 13.9 features covered per application against 7.3. Of 374 self-generated tasks, 85% judged viable and 59% completed by manual labelling. Five crashes found. Cost \$13–\$22 per application, mean \$18.10.
AUITestAgent [7]	GUI <i>function testing</i> driven by a natural-language test requirement; deliberately decouples interaction from verification and extracts verification evidence along three dimensions of the interaction log.	30 authored interaction tasks over 8 popular applications; 77% completion. Verification measured on 40 tasks built from 20 <i>manually injected</i> anomalies: 95% oracle accuracy on correct behaviour, 85% on anomaly detection, 90% recall at a 4.5% false-positive rate. Field-deployed at Meituan: 4 new functional bugs across 10 regression rounds in two months, saving 1.5 person-days per round.
XUAT-Copilot [6]	Automating the <i>test-script generation</i> stage of an existing industrial user-acceptance-testing pipeline; three agents for action planning, state checking and parameter selection, plus reflection.	450 WeChat Pay test cases, all of which had already passed with human-written scripts. <i>Pass@1</i> of 88.55% and <i>Complete@1</i> of 93.03%, against 22.65% and 25.25% for a single-agent variant carrying the same combined prompt.

continued on the next page

Table 2.1: The seven primary systems reviewed (*continued*)

System	Goal and mechanism	Evaluation basis and headline result
AutoDroid [5]	<i>Task automation</i> : combines model commonsense with app-specific knowledge mined by automated dynamic analysis; UI transition memory, memory injection, and query optimisation.	DroidTask benchmark: 158 tasks over 13 applications. With GPT-4, 90.9% action accuracy and 71.3% task completion, 36–40 percentage points above the language-model baselines. Query cost reduced from \$18.76 to \$10.17 per thousand GPT-4 queries by halving prompt length.
AppAgent [3]	<i>Task automation</i> from screenshots alone; an exploration phase writes a knowledge-base document per application, either autonomously or by watching a human demonstration.	45 tasks over 9 applications. Success rate 2.2% with a raw action space, 48.9% with a designed action space and no document, 73.3% with an autonomously explored document, 84.4% from watching demonstrations, and 95.6% from a hand-written document.
AppAgentX [4]	<i>Task automation</i> with an evolutionary memory: repeated low-level action sequences are abstracted into high-level shortcuts, implemented over Neo4j for graph storage and a vector index for retrieval.	AppAgent benchmark (50 tasks, 9 applications), each repeated five times. Success rate 16.9% with no memory, 70.8% with chain-structured memory, 71.4% with memory plus evolution, while average steps fell from 9.1 to 5.7 and tokens per task from 9.26k to 4.94k.

2.1.3 What the Evidence Agrees On

Three findings recur across systems that were built independently, by different groups, on different benchmarks. Because they agree, they can be treated as established rather than as any one paper’s claim.

Persistent memory is the single largest lever, and it is worth more than the model. The ablations are unusually consistent. AppAgentX reports 16.9% success with no memory and 70.8% with chain-structured memory, a four-fold improvement from memory alone, before any other enhancement [4]. AppAgent reports 2.2% with a raw action space and no document, rising to 73.3% once an exploration phase has written one, on the same underlying model [3]. Removing the explored-functionality memory from GPTDroid costs it 51% of its activity coverage, from 0.75 to 0.37, the largest single contribution of any of its modules [10]. AutoDroid’s entire thesis is that app-specific knowledge mined by dynamic analysis, injected into the prompt, is what separates 71.3% task completion from the 31–35% of the unaugmented baselines [5]. This is the strongest single justification available for the knowledge-graph architecture selected in Section 3.3.

Decomposing the agent into roles beats one large prompt. XUAT-Copilot gives the cleanest measurement, because its single-agent variant is constructed by concatenating all three agents’ prompts into one: *Pass@1* falls from 88.55% to 22.65% [6]. The authors attribute the collapse to long prompts inducing forgetting, hallucination and inefficient planning. DroidAgent reaches the same architecture independently with four roles [8], and AUITestAgent makes the separation of interaction from verification its central design claim [7].

The bottleneck is the oracle, not the actuator. Across the seven systems, *interacting* with an application is largely solved: completion rates cluster between 71% and 95%. *Judging* the result is not. The sharpest evidence is AUITestAgent’s baseline, a competent general-purpose model prompted directly: it reaches 0.90 oracle accuracy when the application is behaving correctly and 0.30 when an anomaly is present, and the authors name the cause: “its tendency to output a ‘no anomaly’ judgment” [7]. A verifier that is accurate only when there is nothing to find is not a verifier; it is a silent pass. This single observation motivated two design decisions in this project: the separate investigator role (DR-06) and the prohibition on silent fallbacks (NFR-D2).

A fourth observation is weaker but practically important: **cost is rarely reported, and where it is reported it is substantial.** DroidAgent spends a mean of \$18.10 per application for a two-hour budget and states in its threats to validity that “given the monetary constraints linked to API requests, we could not conduct multiple runs” [8]. That is, the cost of the method prevented the authors from establishing the reproducibility of their own result. AutoDroid treats query cost as a first-class engineering target and halves it by compressing the UI representation [5]. Of the remaining five systems, only AUITestAgent and AppAgentX report token consumption at all.

2.1.4 How This Literature Is Measured, and Why It Reports Almost No Precision or Recall

The most consequential finding of this review is methodological. Across all seven systems, defect-finding ability is established in one of three ways, none of which supports a precision, recall or F_1 figure. Table 2.2 sets out what each regime can and cannot establish.

Table 2.2: The three evaluation regimes in the reviewed literature, and the limit of each

Regime	Used by	What it can and cannot establish
Structural coverage proxies (activity, widget, code, feature)	GPTDroid, DroidAgent, and every pre-LLM baseline	Establishes that more of the application was reached. Cannot establish that anything was <i>checked</i> : an activity can be covered by a control that opens it and a back press, which DroidAgent documents Humanoid doing. Feature coverage is stronger but, absent a specification, must be defined by hand: DroidAgent’s feature list was fixed “until the consensus of three authors”.
Known-crash benchmarks	GPTDroid on Themis [14] plus 73 further applications: 93 applications, 143 bugs	Gives a genuine denominator and therefore a genuine recall, 95 of 143. Its limit is stated by the authors themselves: “same as the benchmark, all bugs are crash bugs.” A crash is the only defect a tool can recognise without knowing what the application was supposed to do, so the entire quantitative bug-finding literature measures crash detection. Functional defects (a wrong total, an order routed to the wrong seller) are outside the benchmark by construction.
Wild-bug counts	GPTDroid (53 new, 35 confirmed or fixed); AUITestAgent (4 in two months at Meituan)	Strong evidence of practical usefulness, and a precision of sorts can be computed from the confirmation rate. Recall is unobtainable in principle: nobody knows how many defects a production application contains, so there is no denominator.

Only two of the seven papers report precision and recall at all, and it is instructive that both had to manufacture their ground truth to do so.

AUITestAgent needed to measure verification ability on commercial applications, and states the obstacle plainly: “as the apps we selected are not open-source and bugs are not common in the online versions of these mature commercial apps, we needed a robust method to effectively measure AUITestAgent’s performance in executing function verification” [7]. Their solution was to *inject* defects: two authors analysed 50 recent historical anomalies at Meituan, distilled them into three categories, and manually created 20 corresponding anomaly manifestations, each paired with one correct and one anomalous interaction trace. Only against that artificial ground

truth could they report 90% recall at a 4.5% false-positive rate.

DroidAgent reports precision 0.72, recall 0.77 and F_1 0.74, but for its *Reflector's self-assessment*, not for defect detection [8]. The measurement is of whether the agent correctly judged that it had completed its own task, scored against three authors' manual labels. It is a measure of self-knowledge, which is valuable and which this project also needs, but it says nothing about how many defects were missed.

The pattern is therefore a chain of dependencies, and it is worth stating explicitly because it determines what this project can and cannot claim: *without a specification there is no oracle beyond a crash; without an oracle there is no labelled ground truth; without ground truth there is no precision, recall or F_1 , only coverage proxies, crash counts, and human consensus*. Every subjective judgement in this literature (DroidAgent's three authors agreeing on a feature list, its manual labelling of 374 tasks, the confirmation of 35 of 53 reported Google Play bugs by their developers) exists to substitute for a ground truth that the absence of a specification made unavailable.

This diagnosis has two direct consequences for the present project. First, it is the strongest argument for the specification-grounded design: ingesting a requirements document supplies exactly the oracle whose absence forces everyone else onto coverage proxies, and makes *requirement coverage* reportable as a first-class result rather than activity coverage as a proxy. Second, it means that this project's own precision and recall cannot be obtained honestly without seeding known defects, in the manner of AUITestAgent. That experiment is specified but was not completed within this session, because the industry partner supplied neither a defect export nor a seeded build (constraint C3), and it is carried into Chapter 7 as the single most valuable piece of outstanding work.

2.1.5 What the Literature Leaves Open

- G1. No specification is ever ingested.** Not one of the seven systems reads a requirements document, builds a requirement graph, or reports requirement coverage. AUITestAgent comes closest by accepting one natural-language test requirement per test, but it holds no persistent model of the requirement set and cannot say which requirements remain untested.
- G2. Memory does not leave the application or the session.** AppAgentX's shortcut chains, AutoDroid's transition memory and GPTDroid's functionality memory are all per-application. No reviewed system transfers what it learned about one application, device profile or platform to another.
- G3. Defect history is never a knowledge source.** None of the seven reads a bug tracker, and none biases exploration toward historically fragile areas, despite defect density being

among the oldest and best-established predictors in software engineering.

- G4. Failures are not attributed.** Outcomes are binary. Nothing in these systems distinguishes “the application is wrong” from “the agent could not drive the application” from “the environment blocked the test”, even though the three demand completely different responses and only the first is a defect report.
- G5. Web and mobile are studied in isolation.** The mobile systems reviewed here share no substrate with their web equivalents, so nothing learned on one platform informs the other.
- G6. Safety is largely unaddressed, and its cost is unmeasured.** Only AutoDroid treats destructive actions as a design problem, introducing privacy replacement and security confirmation and reporting what they cost: action accuracy falls from 92.9% to 89.9% and completion from 75.4% to 69.9% [5]. That roughly 3–6 percentage-point safety tax is the only quantification of this trade-off found in the review, and it is directly relevant to a project constrained to avoid irreversible actions on a production system.

Gaps G1–G4 correspond, respectively, to this project’s specification-grounded planner, its multi-dimensional graph partitioning (ETA-REQ-304), its defect-history source (ETA-REQ-301) and its three-way attribution taxonomy (NFR-D3); G5 corresponds to the shared planner and graph behind the Android and web executors (DR-04); G6 to the enforced safety boundary (DR-07). The combination (a specification graph that says what *should* happen, a runtime graph that records what *does*, and exploration steered by the gap between them) is the position this project occupies in the literature.

2.1.6 Additional Works Surveyed

Beyond the seven systems read in full, the team surveyed a wider set of recent work on generalist GUI agents, visual grounding models and agent benchmarks. Two strands bear directly on the argument of this chapter and are worth reporting; the remainder are noted only briefly.

Benchmark environments with programmatic oracles. The measurement problem of Section 2.1.4 has a partial answer that none of the seven testing systems uses: define success by inspecting durable application or system state after the fact, rather than by asking the agent, or a language model reading the agent’s transcript, whether it succeeded. AndroidWorld builds 116 programmatically checkable tasks across 20 real Android applications, each with an initialisation, a state-based success check and a teardown, and parameterises them dynamically so that a memorised phrasing does not suffice [15]. AppWorld goes further for the class of behaviours that touch stored data: its tasks are graded by database-state unit tests, which accept any valid

solution path while also detecting *unintended collateral changes*, something a transcript-reading judge cannot see at all [16].

Two consequences follow. First, this is a strictly stronger oracle than self-report for the subset of behaviours whose effects are externally observable, and adopting it where possible is recorded as future work in Chapter 7. Second, and more sobering, the scores these benchmarks report are low: the best baseline in AndroidWorld completes 30.6% of tasks [15], and the strongest model in AppWorld solves about 49% of normal tasks and 30% of challenge tasks [16]. Set beside the 71% to 95% completion rates that self-graded systems report (Table 2.1), the gap is itself evidence for this chapter’s central claim: much of the apparent competence of GUI agents is an artefact of letting them mark their own work. A project whose output is a correctness judgement cannot afford to ignore that.

Learning from interactive outcomes. DigiRL argues that static demonstrations do not capture the stochastic behaviour encountered while controlling a real device, and trains a device-control policy by offline followed by offline-to-online reinforcement learning against a vision-language-model evaluator, raising a 1.5B-parameter model from 17.7% to 67.2% success without additional human demonstrations [17]. It is the clearest external support for this project’s premise that execution outcomes must change future behaviour, and it marks the boundary of the approach taken here: this system adapts through graph memory, strategy scores and prompt construction rather than by updating model weights, which keeps it applicable to hosted models it does not control but forgoes the gains a learned policy can reach.

The remainder. The survey also covered visual grounding and screenshot-only action models. These are nearer to this project than to most of the systems in Table 2.1, because this system already combines both signals: the executor and the planner each receive a screenshot alongside the structural description, and a perceptual hash resolves screen identity where the accessibility tree is uninformative, which is the situation constraint C4 describes. The open question they raise is not whether to use pixels but when, since an image carries a cost at every step it is attached to. The survey also covered one recent review of operating-system agents, useful for terminology and for positioning this work as a mobile agent with explicit *testing* responsibilities rather than a general phone assistant. None of these changes the gap analysis of Section 2.1.5.

TODO (team): the three works cited in this subsection were surveyed from their published abstracts and metadata by a team member, not read in full by the author of this chapter, and their references.bib entries use first-author-et-al form. Verify all three against the published versions before submission, and either read the visual-grounding and survey works and cite them properly or leave them uncited as they are here.

2.2 Existing Solutions and Technologies

Section 2.1 surveyed the research. This section surveys what is actually deployable today, including what the industry partner already runs.

Table 2.3: Deployable technologies in the automated GUI testing space

Category	Representative tools	Strengths and limitations
Scripted UI automation frameworks	Appium, Espresso, UI Automator, XCUITest, Selenium, Playwright	The industry default. Deterministic, fast, and integrated with continuous integration. They are authoring frameworks, not test designers: every case is written and maintained by a person, and a user-interface redesign invalidates the locators wholesale. They confirm known behaviour and discover nothing.
Device farms and execution infrastructure	Firebase Test Lab, AWS Device Farm, BrowserStack, Sauce Labs	Solve the device-fragmentation half of the problem: many real devices, on demand. They execute whatever tests they are given and contribute nothing to deciding what to test. Complementary to this project rather than competing with it.
Commercial “AI-assisted” testing	Testim, mabl, testRigor, Functionize, Appliflow	Two genuine capabilities: self-healing locators, which re-identify a control after a redesign, and visual regression, which diffs rendered screens. Both reduce maintenance cost on an existing suite. Neither authors new tests from a specification, and the visual approach has no semantic oracle: it detects that a screen changed, not that the change was wrong.
Open-source automated explorers	Monkey [1], DroidBot [11], Humanoid [12], Stocat, Ape, Fastbot	Free, unattended, and effective at finding crashes. Optimise structural coverage; no functional oracle; the output is a crash log rather than a test case a person can read or re-run deliberately.

continued on the next page

Table 2.3: Deployable technologies in the automated GUI testing space (*continued*)

Category	Representative tools	Strengths and limitations
Research agents	The seven systems of Table 2.1	Demonstrate the capabilities discussed above, but are research artefacts: several provide no replication package (DroidAgent had to re-implement GPTDroid for comparison) and none ingests a specification or defect history.
The partner's existing system	The static-knowledge generator described in ETA-TR-2026-001 §2 [2]	Already ingests an SRS and a Figma export into a Neo4j knowledge graph and runs a multi-stage retrieval-planning loop to generate non-duplicate test cases. Its documented limits are the starting point of this project: retrieval is token-overlap scoring only; execution memory is a verdict and a free-text note per run; and there is no path memory, no defect correlation and no cross-session learning.

The gap

Placing the research and the products side by side yields a clean statement of the gap. Commercial tools have adoption, reliability and integration but no test design: they automate execution and maintenance of tests a person wrote. Research agents have test design but no specification, no durable cross-application memory, no defect history and no fault attribution: they explore, and they report crashes. The partner's existing system has the specification and the graph but no learning: it is a static-knowledge generator, and a second run knows exactly what the first one knew.

This project addresses the intersection: a system that keeps the partner's specification grounding, adds the research agents' autonomous execution, and closes the loop between them by persisting every execution outcome as structured knowledge that measurably changes the next proposal, while adding the two things neither side has, namely sound fault attribution and an enforced safety boundary.

2.3 Market and User Research

Who would use it

The primary users are quality-assurance engineers in organisations that ship graphical applications on a short release cadence, the population that today performs manual exploratory testing, maintains scripted suites, or both. The secondary users are the application developers who consume the output, whose requirement is not more reports but fewer false ones. A third group, relevant to a Bangladeshi context, is the smaller application-development firm that cannot staff a dedicated QA team at all and currently ships with ad-hoc manual checks; for them the alternative to an automated exploratory agent is not a cheaper tool but no systematic testing.

Evidence of demand

Four independent sources of evidence were available to this project.

The industry partner’s own commissioning document. Samsung Research and Development Institute Bangladesh specified an eight-family enhancement programme with a stated effort estimate of 17–23 weeks and a phased dependency ordering [2]. An organisation does not scope four to six months of engineering against a capability it does not need; the document is the strongest single demand signal available to the project, and it is specific about *which* capability, learning across executions, is the valuable part.

Field deployments reported in the literature. Two of the reviewed systems were deployed in production organisations rather than only evaluated. AUITestAgent runs in Meituan’s weekly regression process for a video business line, where its authors report a saving of 1.5 person-days per regression round and four new functional bugs over two months [7]. XUAT-Copilot has “launched in the formal testing environment of WeChat Pay mobile app”, where its authors report that it “saves a considerable amount of manpower in the daily development work” [6]. Both are large payment or commerce platforms committing production processes to language-model agents, which is a stronger signal than any published forecast.

Evidence about what users want the output to look like. The developer study cited by DroidAgent is directly informative for the interface design: the preferred form for an automatically generated test is *natural language*, not a test-API script, and it should carry expected outputs, the steps of the use case, and the specific application feature under test [8]. This project’s generated test cases carry exactly those elements, and the PDF run report (DR-08) exists because a result that can only be read from a log does not reach the person who must act on it.

Direct engagement with a second application. A production Bangladeshi agricultural-

commerce application was supplied by the supervisor as a second target, with live users and a real payment and one-time-password flow. Its inclusion was what forced the project's generality requirement (DR-03) and its safety boundary (DR-07) from aspirations into enforced constraints.

What users do instead, and what it costs them

The present alternatives are manual exploratory testing, scripted regression suites, and unattended crash hunting with Monkey-class tools. The first is the most effective and the least scalable; the second scales but only over behaviour someone already anticipated; the third is nearly free and finds only crashes. The measured saving of 1.5 person-days per regression round for a *single* business line [7] gives an order of magnitude for the manual effort in play.

TODO (team): if a structured interview with the partner's QA engineers or a practitioner survey is conducted before submission, report its method, sample and findings here; that would upgrade this section from secondary evidence to primary. Also add any market-size figure the team can cite properly.

2.4 Feasibility Study

Technical feasibility

Feasible, and demonstrated rather than argued: the delivered system executes on a physical Android device and in a browser, and the individual capabilities have independent precedent in the literature. The two technical risks are real and were both encountered. The first is applications whose accessibility tree is thin: DroidAgent reports the same failure mode, naming applications “whose widgets do not have any textual properties” and single views containing distinct interactive sub-regions as the cases where its coverage collapses [8], which is exactly the behaviour of the cross-platform-toolkit application in constraint C4. The second is non-determinism, which is intrinsic and is managed by evaluating over distributions of runs rather than single runs (constraint C8).

Economic feasibility

This is the dimension on which the project's position is strongest, and it needs stating carefully because the units differ. DroidAgent's cost is reported per application for a fixed two-hour exploration budget: \$13–\$22, mean \$18.10 [8]. AutoDroid reports per query: \$10.17–\$18.76 per thousand GPT-4 queries after optimisation [5]. This project measures per *executed test case*, and its figure, reported with its derivation in Chapter 5, is of the order of one United States cent, because roughly nine tenths of each execution's work is performed on the device rather than in

the model, and because the executor and investigator roles were deliberately moved to cheaper model tiers once the quality cost of doing so had been measured.

The comparison is therefore indicative rather than like-for-like: a two-hour DroidAgent campaign and one of this project's test executions are different units of work. What the comparison does establish is that per-test cost is a tractable engineering variable rather than a fixed property of the approach, and that the specific failure DroidAgent reports (being unable to repeat runs because of API cost, and having to record that as a threat to validity [8]) is avoidable. For a student project on a fixed prepaid balance (constraint C2), that difference is the difference between running a campaign and not running one.

Operational feasibility

The system is operable by one engineer: a target is configured as a validated JSON profile through the dashboard rather than as a code change (DR-09), a campaign is launched and watched from the same surface, and its result is exported as a PDF a non-participant can read (DR-08). Two operational prerequisites remain and are not removable: a physical device or emulator with the companion accessibility service installed, and disposable test accounts provisioned in advance for any application behind a sign-in. The deployment model is the genuine operational boundary: the system is scoped to run as a local, developer-operated set of processes on a trusted host (constraint C7), so adoption into an organisation's shared infrastructure would require a hardening pass that this project did not take on.

Legal and regulatory feasibility

Feasible under the instruments analysed in Section 3.2, subject to the controls stated there: dedicated disposable test accounts rather than production user data, destructive and communicative actions blocked at the configuration layer, application content treated as untrusted data in every prompt, and credentials kept out of the planner path entirely. No regulatory obstacle to the project as scoped was identified.

Conclusion and the assumptions it rests on

The project is feasible on all four dimensions. Its feasibility depends on four assumptions, each of which held during this work and each of which would need re-examination for a different target: that the application under test exposes an accessibility tree or a DOM; that disposable test accounts can be provisioned in advance for authenticated flows; that a hosted model API remains available at roughly current prices; and that the organisation accepts a reported defect as

a *candidate* requiring human confirmation rather than as a verified fault. The last of these is not a technical assumption but an organisational one, and on the evidence of Section 2.1.4 it is the assumption the entire field currently operates under.

Chapter 3

Requirements Analysis

3.1 Project Objectives and Constraints

3.1.1 The Primary Requirement: From Directed to Exploratory Testing

Every other requirement in this chapter is downstream of one decision taken by the industry partner, and it is worth isolating before the detail begins. The partner’s review of the preceding work asked for a move away from *directed functional testing*, in which an agent is handed a test objective and carries it out, towards *exploratory testing*, in which the agent decides for itself what is worth testing next. ETA-TR-2026-001 [2] records the target as “an autonomously learning, self-optimising exploratory test engineer” and the governing principle as *every execution cycle must make the agent smarter than the last*.

Read as a requirements statement rather than a slogan, that sentence has four consequences that shape everything in Sections 3.1.3 and 3.1.4.

- (a) **The test objective moves from input to output.** Nothing outside the loop may supply a test case. This is what makes the planner, rather than the executor, the component on which the system’s quality depends, and it is why supplying individual objectives is explicitly out of scope (Section 1.4).
- (b) **Memory becomes mandatory rather than an optimisation.** An exploratory agent with no persistent state re-treads the same ground every session, so “smarter than the last cycle” is unsatisfiable without durable knowledge. This is why five of the partner’s eight requirement families (ETA-REQ-301, 302, 303, 306 and 307) are, at bottom, statements about *remembering*.
- (c) **The oracle must be derived, and must be able to abstain.** A directed test carries its expected result; an exploratory one must infer what correct behaviour is from the specification and decline to judge when the specification is silent. The two failure modes of a derived oracle, a fabricated defect and a silent pass, are what NFR-D2 and NFR-D3 exist to prevent.

- (d) **Success must be measured differently.** Task-completion rate is the natural metric for a directed agent, because the task was given. For an exploratory agent the comparable questions are how much of the requirement set was covered, how many executions produced a verdict about the application rather than about the agent, and what each of those cost, which is why the optimised quantities in Section 1.2 are coverage, autonomy and cost per test rather than task success.

The difficulty this buys is set out as D1–D6 in Section 1.1.2 and is not restated here. It is, however, the reason several of the derived requirements in Table 3.3 exist at all: a directed agent would need neither a three-way fault attribution nor an investigator role, because a supplied objective already answers most of the questions those mechanisms are there to resolve.

3.1.2 Stakeholders and Their Expectations

The problem came from an industry partner, which makes the stakeholder map concrete rather than hypothetical. Table 3.1 lists each group, what it expects and, where it exists, the tension between that expectation and another group's.

Table 3.1: Stakeholders, expectations and conflicts

Stakeholder	Expectation	Conflict with another stakeholder
Industry partner (Samsung R&D Institute Bangladesh)	A move from directed functional testing to genuinely exploratory testing; an agent that accumulates experiential knowledge and transfers it across device profiles, platforms and applications; delivery against ETA-TR-2026-001.	Self-directed exploration conflicts directly with the QA lead's demand that every reported result be reproducible and traceable, and transfer implies generalising beyond what was observed.
Quality-assurance engineers	Reduced manual effort; runs that can be reproduced, audited and explained; a report that can be handed to a developer.	Reproducibility conflicts with the exploratory, non-deterministic behaviour that gives the agent its value (difficulty D5).
Application developers	A low false-positive rate. Every incorrectly reported defect costs investigation time and erodes trust in the tool.	Conflicts with maximising defect-discovery rate: a more aggressive oracle finds more real defects and more spurious ones.

continued on the next page

Table 3.1: Stakeholders, expectations and conflicts (*continued*)

Stakeholder	Expectation	Conflict with another stakeholder
Academic supervisor and examiners	A defensible empirical evaluation with an honest account of limitations; evidence of engineering judgement, not only of a working artefact.	Conflicts with the partner's delivery pressure, which rewards breadth of features over depth of validation.
End users of the application under test	That testing never damages live data, charges a payment method, or sends a message on their behalf.	Conflicts with coverage: the safest agent is one that never acts.
The project team	A system that one member can operate without the author of a subsystem present.	Conflicts with delivery speed.

The two conflicts that shaped the design most are the third and the fifth. The false-positive conflict is resolved by making fault attribution a first-class, single-definition concept (Section 3.1.4, NFR-D3) so that an agent limitation can never be silently reported as an application defect. The safety conflict is resolved by declaring irreversible flows out of scope and enforcing that declaration in configuration rather than in prose (Section 3.1.5, C5).

3.1.3 Functional Requirements

The functional requirements come from two places. The first eight families are the partner's, taken verbatim in intent from ETA-TR-2026-001 [2] sections 3.1–3.8. The remainder (prefixed DR, for *derived*) were identified by the team during the work and agreed with the supervisor; they exist because the partner's document assumes a single, fully documented Android application, and that assumption did not survive contact with a real target.

Table 3.2 lists the partner's requirements with the module that implements each and an honest status. Table 3.3 does the same for the derived requirements.

Table 3.2: Partner requirements (ETA-TR-2026-001) and their realisation

ID	Requirement	Realisation	Status
ETA-REQ-301	Defect history as a knowledge-graph source: ingestion, graph model, defect-weighted retrieval, defect summary, prompt injection, test-to-defect traceability.	<code>ingestion/defect_loader.py</code> , <code>rag_api/defects.py</code> , <code>planner/sources/defects.py</code> ; endpoints <code>/ingest/defects</code> , <code>/defects/summary</code> , <code>/defects/prone-areas</code> , <code>/defects/context</code> .	Implemented. Exercised on synthetic defect data only; the partner supplied no real defect export (constraint C3).
ETA-REQ-302	Execution-path memory as a navigation tree: path recording, shortest-path optimisation, branch retrieval, incremental extension, failed-path avoidance, cross-test reuse.	<code>rag_api/navtree.py</code> , <code>planner/sources/navtree.py</code> ; endpoints <code>/navtree/record-path</code> , <code>/navtree/retrieve-path</code> , <code>/navtree/failed-paths</code> , <code>/navtree/stats</code> .	Implemented.
ETA-REQ-303	Incremental experiential learning: execution logs, error-pattern mining, strategy memory, coverage heat map, session continuity, knowledge decay.	<code>rag_api/learning.py</code> ; endpoints <code>/execution/log</code> , <code>/execution/logs</code> , <code>/execution/error-patterns</code> , <code>/coverage/heatmap</code> , <code>/strategy/memory</code> and the four <code>/session/</code> endpoints. Requirement knowledge is versioned rather than overwritten: <code>/srs/drift</code> reports the rules added, changed or removed when a specification is re-ingested, and <code>/business-logic/rules</code> exposes each extracted validation rule with the source chunk it came from. Extraction itself supports best-of- N sampling adjudicated by a separate judge model.	Implemented.
ETA-REQ-304	Multi-dimensional graph partitioning by profile, platform and application, with dimensional retrieval filtering and cross-dimensional transfer.	<code>rag_api/dimensions.py</code> ; endpoints <code>/dimensions/register</code> , <code>/dimensions/list</code> , <code>/dimensions/transfer-suggestions</code> .	Implemented. The application axis is exercised; cross-dimensional transfer is not yet validated.

continued on the next page

Table 3.2: Partner requirements (ETA-TR-2026-001) and their realisation (*continued*)

ID	Requirement	Realisation	Status
ETA-REQ-305	Self-healing and adaptive execution: failure classification, per-category retry, failure context injected into the retry prompt.	<code>clients/executor_runner.py</code> (Android) and <code>web_player/failures.py</code> (web), sharing one category vocabulary defined in <code>settings.py</code> .	Implemented on both platforms.
ETA-REQ-306	Regression risk scoring per feature area, risk-weighted prioritisation, risk API.	<code>rag_api/risk.py</code> , endpoint <code>/risk/scores</code> ; the [RISK] priority line in <code>planner/coverage.py</code> .	Implemented.
ETA-REQ-307	Test-case quality metrics and feedback loop: per-test effectiveness, generation-quality feedback, semantic duplicate detection.	<code>rag_api/metrics.py</code> ; endpoints <code>/tests/effectiveness</code> , <code>/tests/dedup-check</code> . Embedding-cosine dedup is called from both planner proposal paths, behind a Jaccard pre-filter in <code>planner/textutil.py</code> .	Implemented.
ETA-REQ-308	Anomaly detection over execution patterns: failure-rate spikes, duration regressions, new error types, path instability.	<code>rag_api/anomalies.py</code> ; endpoints <code>/anomalies/detect</code> , <code>/anomalies</code> .	Implemented. Validated on synthesised execution histories.

Table 3.3: Requirements derived by the team, beyond ETA-TR-2026-001

ID	Requirement and why it was needed	Realisation
DR-01	No knowledge source is a hard dependency. The partner's document presumes an SRS <i>and</i> a Figma export. Most real targets have one or neither, and a system that fails without both is not deployable. Adding a source must be additive; removing one must degrade the output, not break it.	A source registry with a common interface: <code>planner/sources/ (registry.py, base.py)</code> with nine registered sources, consulted by the planner instead of a hard-coded source list.

continued on the next page

Table 3.3: Requirements derived by the team (*continued*)

ID	Requirement and why it was needed	Realisation
DR-02	Learn the application’s structure from the running application. With no design export, the only ground truth is what the screen shows. The agent must build its own map of screens and transitions, and must correctly decide whether a screen it is looking at is one it has seen before.	The Live App Model: <code>ingestion/app_state.py</code> computes a structural signature and the identity cascade, which ends in a perceptual hash over the screenshot and is consulted only where the accessibility tree is too thin to decide on structure alone. Endpoints <code>/liveui/observe</code> and <code>/appmodel/graph</code> store and expose the state–transition graph, and each state keeps its screenshot, so both the planner and the operator can see the screen a test refers to.
DR-03	Application-agnostic by construction. An agent that knows about one application is a script. No application name, package, screen name or domain vocabulary may appear in code or in a prompt template.	All application-specific values are injected from configuration through <code>settings.py</code> block builders; a repository-wide generality audit script enforces the rule.
DR-04	A second execution platform. Restricting the agent to Android would have left the learning machinery unvalidated against a genuinely different user-interface technology, and the partner’s own multi-platform requirement (ETA-REQ-304) would have had only one platform to partition.	<code>web_player/</code> : a Playwright-driven executor with its own snapshot, action, oracle and failure-classification modules, sharing the planner, the graph and the attribution vocabulary. It carries the operational guards a live site needs: a single-run lock so two campaigns cannot interleave against one target, and a pause-and-notify handoff when a CAPTCHA blocks a flow, so the run waits for a person rather than recording a false failure.
DR-05	Evidence integrity. Two failure modes were observed early and are unacceptable in a tool whose output is a correctness claim: a language-model outage being recorded as a defect in the application, and a silent fallback hiding the fact that a component had degraded.	A single three-way attribution taxonomy (<code>APP_FAULT</code> , <code>AGENT_FAULT</code> , <code>ENV_FAULT</code>) defined once in <code>settings.py</code> and consumed everywhere; a cross-process degradation register in <code>observability/</code> that records every fallback taken.
DR-06	Turn trajectories into durable, atomic knowledge. Raw execution prose does not compose: inlining each run’s report consumed the majority of the next generation prompt and restated the same discovery repeatedly.	An investigator role (<code>rag_api/findings.py</code> , endpoint <code>/execution/evaluate</code>) that reduces a trajectory to deduplicated findings carrying an independent-observation count.

continued on the next page

Table 3.3: Requirements derived by the team (*continued*)

ID	Requirement and why it was needed	Realisation
DR-07	Enforced safety boundary. The declaration that registration, one-time passwords, payment and account deletion are out of scope must be machine-enforced, not a note in a prompt.	Per-target blocked control texts and blocked URL patterns in the target profile schema; an out-of-scope requirement filter that withholds such requirements from the planner's citable list.
DR-08	An operator surface. A campaign that can only be understood by reading a log is not usable by the QA engineers who are the intended users.	A React dashboard with live session, coverage, app-model and trace panels; an exportable PDF run report (<code>gateway/report_api.py</code>) whose numbers are computed from the graph while the language model supplies prose only; knowledge sources uploaded and ingested per target from the same surface; and the graph itself inspectable through visualisation and query endpoints.
DR-09	Configuration as data. Process-global settings made every new target a code change.	A target profile system (<code>targets/</code>) with a validated JSON schema, a loader, and gateway endpoints to list, validate, save, ingest and run a profile.
DR-10	Measured, not estimated, cost. A budget-bound project must know its unit economics.	Per-execution accounting reconciled against the provider's billing record; reported in Chapter 5.
DR-11	The planner must be able to investigate, not only receive. A fixed context block has to guess in advance what the planner will need, and it cannot check a claim while forming it. The consequence was measurable: screen names the planner invented matched a real observed screen about 16% of the time, and invented requirement identifiers reached the device.	Two interchangeable planner modes behind one output contract (<code>PLANNER_MODE</code>). The default assembles a fixed context; the alternative gives the planner nine graph tools (requirement search, untested requirements, screen lookup, screen listing, findings summary and listing, open questions, coverage, navigation paths) and lets each tool result enter its conversation. A tool whose knowledge source is disabled is never registered, so an absent source cannot mislead it. Proposals pass a server-side gate that checks the screen exists, the requirements exist, the test is not a semantic duplicate and the preconditions are reachable, returning an actionable reason on rejection. The executor is unaffected by the choice, so the two can be compared on the same campaign.

3.1.4 Non-Functional Requirements

Two non-functional requirements are the partner's; the rest were derived alongside the DR requirements above.

- **NFR-001 (partner). Retrieval latency.** All retrieval endpoints shall respond within 500 ms at the 95th percentile for typical queries, on a graph of up to 100,000 nodes.
- **NFR-002 (partner). Graph scalability.** The system shall support up to 10,000 test cases, 5,000 defects and 50,000 navigation nodes per project without performance degradation.
- **NFR-D1. Cost ceiling.** The marginal cost of one executed test shall be small enough that a campaign is not budget-limited at the scale of a few hundred tests.
- **NFR-D2. Observability.** Every degradation, fallback and skipped step shall be recorded and retrievable across process boundaries. A silent pass is treated as a defect in the harness.
- **NFR-D3. Attribution soundness.** No agent limitation or environmental block may be reported as an application defect. The taxonomy has exactly one definition in the source tree.
- **NFR-D4. Safety.** No irreversible action on the application under test, enforced by configuration.
- **NFR-D5. Reproducibility.** A campaign shall be reconstructible from persisted artefacts (graph state, execution logs, screenshots and the exact prompt sent) without re-running it. Given difficulty D5 this is deliberately *reconstruction*, not replay.
- **NFR-D6. Confidentiality of test credentials.** Credentials for the test account shall not be committed, and shall not reach any component that does not need them; in particular the planner shall receive the account *role* but never the secret.

3.1.5 Constraints

C1. Time.

One academic session, with five part-time team members who also carried a full course load. This ruled out approaches requiring model training or a large hand-labelled corpus.

C2. Budget.

A fixed, small prepaid balance on a hosted model-inference provider. There was no institutional compute grant. Cost per test therefore became a design constraint rather than a reporting afterthought, and directly motivated moving work from the flagship model tier to cheaper tiers where the quality loss was acceptable.

C3. Data availability.

The partner did not supply a defect export or a seeded-defect build. ETA-REQ-301 could

therefore be implemented but only validated against synthetic defect data, and precision and recall against a known ground truth could not be measured. This is recorded as a limitation rather than worked around.

C4. Target application.

The application supplied for evaluation is a production system with live users. It is built with a cross-platform toolkit that exposes no stable resource identifiers, reports a single activity on every screen, and does not expose text through the accessibility tree, all of which the element-location strategy had to accommodate.

C5. Irreversible flows.

Account registration, one-time-password verification, payment and account deletion cannot be exercised against a production system. Test accounts were provisioned in advance and the corresponding requirements marked as preconditions already satisfied.

C6. Device access.

Testing requires a physical device or emulator reachable from the developer host, with the mobilerun companion application installed. That companion supplies both the accessibility service the executor drives and an input method that makes text entry reliable; without it the executor falls back to shell input, which corrupts multi-field forms. Emulator use was additionally limited by the target's dependence on a telephone-number sign-in.

C7. Deployment.

The system is scoped to run as a local, developer-operated stack of four processes on a single trusted host. Hardening for shared-network or multi-tenant deployment is outside the project's scope, and this bounds the deployment claims made in Chapter 4.

C8. Non-determinism.

Language-model outputs are not reproducible across calls. Any evaluation claim must therefore be made over a distribution of runs, not a single run.

3.2 Regulatory Requirements and Standards

No single standard governs an autonomous, model-driven test agent. The applicable instruments are therefore of three kinds: testing standards that supply vocabulary and process; security and privacy instruments that constrain how the system handles the application's data and its own credentials; and platform or partner policies. Table 3.4 states each, why it applies, and how the design complies.

Table 3.4: Standards and regulatory instruments, applicability and compliance

Instrument	Why it applies	How the design complies
ISO/IEC/IEEE 29119 (Software testing, Parts 1–4) [9]	Supplies the normative vocabulary for test process, documentation and techniques, and the definition of a test oracle.	Test cases carry the 29119 documentation elements: objective, preconditions, steps, expected result, and traceability to a requirement. Coverage is reported against the requirement set. <i>Deviation, declared:</i> 29119 assumes test design is performed by people before execution; here it is performed by an agent at run time, so the “test design specification” is the generated artefact itself rather than a prior document.
ISO/IEC 25010 (Systems and software quality models) [18]	Gives the quality characteristics against which the application under test, and the agent itself, are judged.	Functional suitability and functional correctness are the characteristics the oracle targets; reliability and usability observations are recorded as findings but not asserted as defects.
ISO/IEC/IEEE 29148 (Requirements engineering) [19]	The system ingests a requirements specification and cites requirement identifiers; it therefore depends on that document being well-formed.	The ingestion pipeline extracts atomic, uniquely identified requirements and links every derived artefact to the source chunk it came from, preserving bidirectional traceability as 29148 requires. Requirements that are not verifiable as written are flagged rather than silently dropped.
OWASP Top 10 for Large Language Model Applications [20]	The agent feeds untrusted application content (on-screen text, DOM content, page titles) into models that then decide what action to take, which is the textbook prompt-injection surface (LLM01). It also handles account credentials (LLM06).	Every prompt that carries observed application content labels it explicitly as untrusted data that must never be interpreted as instructions; this instruction is present in the executor, investigator and evidence-reviewer prompts. For LLM06, the test-account secret is passed only to the executor goal and never to the planner, which receives the account role alone.

continued on the next page

Table 3.4: Standards and regulatory instruments (*continued*)

Instrument	Why it applies	How the design complies
OWASP Application Security Verification Standard	The stack exposes HTTP services that can ingest files and reset a project database.	Services bind to loopback by default and support API-key authentication, and the deployment model assumes a single trusted host (constraint C7). Uniform authentication across every diagnostic handler, and containment checks on ingestion paths, are identified as the prerequisites for any deployment beyond that host.
Android accessibility APIs and WCAG 2.1 (by analogy)	The Android executor drives the application through the platform accessibility tree; the web executor uses the accessibility-role-annotated DOM. The agent can only interact with what is exposed to assistive technology.	Element location is built on accessible roles, names and content descriptions rather than pixel coordinates. A consequence worth stating: a control the agent cannot see is also a control a screen-reader user cannot see, so the agent's element-location failures are themselves an accessibility signal.
Data-protection principles (GDPR Articles 5 and 25; Bangladesh Personal Data Protection Bill)	Testing a production system risks touching real user data.	Purpose limitation and data minimisation are applied by construction: dedicated disposable test accounts, no ingestion of production user records, and no persistence of raw personal data in the knowledge graph. Destructive and communicative actions are blocked at the configuration layer.
Partner confidentiality policy and third-party licensing	The partner's requirements document is not public; the project uses open-source libraries and commercial model APIs.	No partner-confidential material is reproduced in this report beyond requirement identifiers and titles used for traceability. Third-party licences are listed in Chapter 6, and model-provider terms of service govern the hosted inference used.

3.3 Formulation of Solutions

Five architectures were considered, each of which could plausibly have met most of the functional requirements. They are described below and then compared against criteria drawn directly from Sections 3.1.3 to 3.1.5.

Alternative A. Scripted automation with a page-object model. Encode each requirement as an Appium or Espresso script over a page-object layer, and run the suite on every build. Mature tooling, fully deterministic, trivially reproducible.

Alternative B. Random and search-based exploration. Drive the application with generated events, using a search heuristic (activity coverage, code coverage) to guide the generator. Requires no specification and no per-application authoring.

Alternative C. Classical model-based testing. Hand-author a finite-state model of the application and derive test paths from it by graph traversal. Strong coverage guarantees over the model.

Alternative D. Stateless language-model prompting. Send the specification and a screenshot to a large language model on each round and ask it for the next test case. No persistent memory; each call is independent.

Alternative E. Knowledge-graph-backed multi-role agent (selected). A planner, an executor and an investigator, each a separate stateless language-model call, communicating only through a persistent property graph that is grown from documents, from defect history and from the agent's own observations of the running application.

Conclusion and justification of the selected solution

The comparison separates the alternatives on two rows, and they are the two that encode the partner's actual request. The first row disqualifies A and C outright: a scripted suite and a hand-authored state model both require a person to decide what is tested, which is the directed-testing regime the partner asked to move away from. The fifth row then disqualifies B and D: none of the four carries anything forward from one execution to the next. A re-runs the same scripts, B re-randomises, C traverses a model only a human can update, and D begins each round with no memory of the previous one. Since *every execution cycle must make the agent smarter than the last* is the stated objective, that row alone eliminates four of the five.

Table 3.5: Evaluation of alternative solutions against the project’s criteria. Legend: ✓ meets, ~ partially meets, × does not meet.

Criterion (source)	A	B	C	D	E
Selects its own test objective (Section 3.1.1)	×	✓	×	✓	✓
Requirement traceability and coverage reporting (O2, ETA-REQ-307)	✓	×	~	~	✓
Oracle strength beyond crash detection (Section 1.2)	✓	×	✓	~	✓
Discovers behaviour nobody anticipated (O3)	×	✓	×	✓	✓
Learns across runs; later rounds better informed (O4, ETA-REQ-302/303)	×	×	×	×	✓
Tolerates user-interface change without re-authoring (C4)	×	✓	×	✓	✓
Per-application setup cost (C1)	×	✓	×	✓	~
Works with a partial or absent specification (DR-01)	~	✓	×	~	✓
Cross-platform and cross-application transfer (ETA-REQ-304)	×	×	×	×	✓
Marginal cost per test (C2, NFR-D1)	✓	✓	✓	×	~
Reproducibility and auditability (NFR-D5)	✓	~	✓	×	~
Sound fault attribution (NFR-D3)	✓	×	✓	×	✓

Among the remainder, the choice is between D and E, which differ in exactly one respect: whether the agent has a persistent, queryable memory. Adding that memory is what makes requirement traceability, navigation reuse, risk scoring, duplicate suppression and cross-application transfer expressible at all: five of the partner’s eight requirement families are statements about *remembering*, and are unimplementable without it. A graph, rather than a document store or a vector index alone, was selected because the relationships are the payload: a test case covers a requirement, an execution log belongs to a test case, a navigation node leads to a UI state, a defect concentrates in a feature area. Those are traversals, and a property graph answers them directly while also supporting the vector indexes that dense retrieval requires.

Alternative E is not selected without cost, and the two cells where it is only partial are real. Its marginal cost per test is non-zero, because every round makes metered model calls, addressed by moving the bulk of the work to cheaper model tiers and by measuring the unit cost rather than assuming it (DR-10). Its reproducibility is weaker than A’s, because the planner and executor are non-deterministic, addressed by persisting the exact prompt, the trajectory, the screenshots and the resulting graph state for every execution, so that a run can be *reconstructed and audited* even though it cannot be *replayed identically* (NFR-D5). Per-application setup cost is higher than B’s or D’s, because a target profile must be written, addressed by DR-09, which reduces that cost to editing a validated JSON document through the dashboard rather than changing code.

The selected architecture is therefore E, with A retained as a complementary technique rather than a rejected one: the test cases this agent discovers and confirms are exactly the material from which a conventional deterministic regression suite should later be written. Chapter 4 details the realisation of E.

Chapter 4

System Design and Implementation

4.1 Functional Design and Sub-problem Partitioning

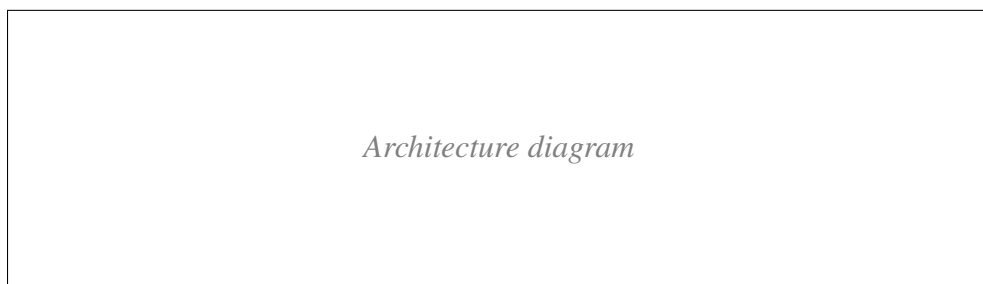


Figure 4.1: System architecture

Partition the problem into sub-problems or modules; explain each one's responsibility and how they interact. Show where the standards and codes of practice from Section 3.2 and any health, safety, or environmental considerations shaped the design.

4.2 Design Refinement and Simulation

Present the detailed design using the views that fit the project (ERD, class, sequence, DFD, API, wireframes, circuit, model). Include any design calculations or estimates made (e.g. expected load or storage), the analyses, prototypes, or simulations used to verify the design against the requirements, and how the design was refined to facilitate implementation.

4.3 Application of Tools

Justify the selection of the important tools and technologies and state their relevant limitations.

4.4 Implementation

Describe how the key modules were implemented and the development process followed (repository, version control, code review, CI/CD).

4.5 Deployment

Describe how the system is deployed and operated: the target environment and infrastructure, configuration and release process, monitoring and backup, and who operates and maintains it. State the computing resources the deployment consumes; their environmental impact is assessed in Section 6.4.

Chapter 5

Investigation and Evaluation

5.1 Experimental Investigation

Present the investigation carried out to substantiate the solution. State the research questions or hypotheses, the method and experimental design, the datasets or measurements used and how they were obtained, the baselines or points of comparison, and the metrics. Analyse and interpret the data, and synthesise the findings into conclusions that are supported by the evidence. Projects with a substantial research and development component should treat this as the core of the chapter; for development-focused projects, report the investigation that validated the main design decisions. This is the experimental work on the built system; the background research is in Chapter 2.

5.2 Performance Evaluation

Evaluate the system against every requirement and objective. Describe the setup, tools, inputs, and acceptance criteria; present the results (tests, measurements, security checks, user or partner acceptance, experiments as appropriate); state which requirements are met and clearly explain each deviation.

5.3 Deviations and Design Revisions

For each deviation, give the technical analysis of its cause and the design revision made to achieve compliance, or explain why it was not revised.

Chapter 6

Project Management, Teamwork, and Impact

6.1 Project Management and Finance

6.1.1 Planning and Risk Management

Present the plan: phases, milestones, task distribution, and schedule, noting where actual progress differed. List the major risks and how they were mitigated.

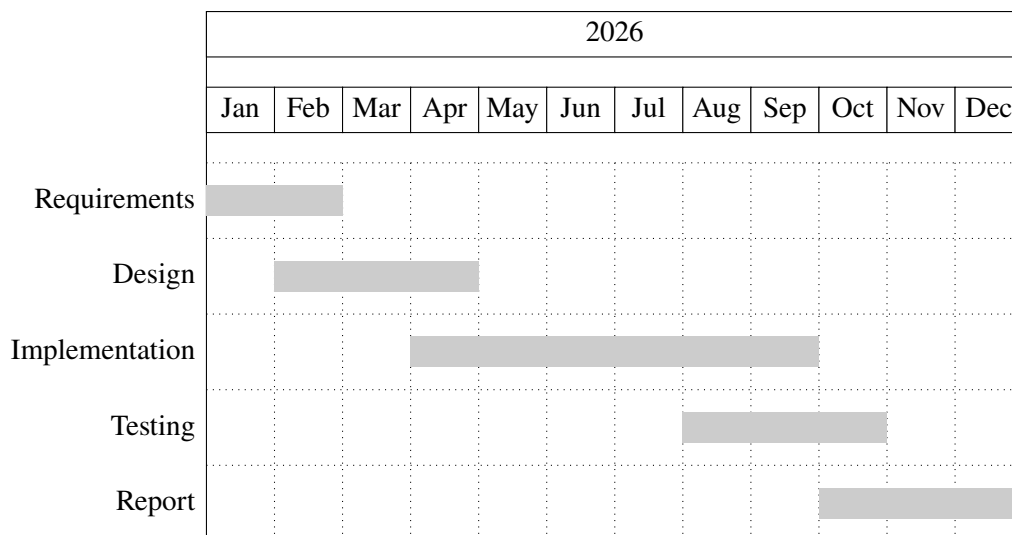


Figure 6.1: Project schedule

6.1.2 Budgeting and Resource Identification

Identify the resources required (people, hardware, software, cloud and API services, data) with an itemised budget.

6.1.3 Economic Analysis and Financial Prospecting

Evaluate the financial prospect of the project with clear interpretation: development and operating cost, the potential users or market established in Section 2.3, and how the solution could generate value or revenue.

6.1.4 Sustainability in Business and Commercialisation

Explain how the solution could remain viable after the course: the commercialisation or deployment path (e.g. with the industry partner) and who would maintain and operate it.

6.2 Individual and Teamwork

6.2.1 Participation and Contribution

For each member, give the role, specific contributions, participation in discussions and decision-making, and whether responsibilities were completed on time.

6.2.2 Collaboration and Conflict Resolution

Describe how the team collaborated and kept a constructive atmosphere, and how any conflicts were identified and resolved.

6.2.3 Leadership and Direction

Explain how direction toward the goals was provided, how members' views were listened to, how the team planned for improvement after setbacks, and how members were motivated.

6.2.4 Multidisciplinary Engagement

Explain the multidisciplinary nature of the project and the team's participation in activities outside its own discipline: work with the industry partner, domain experts, end users, or other departments.

6.3 Impact on Society

This system does not reach the public directly. It tests other people's software, so its effect on society travels through two channels: the quality of the applications that reach users, and the

work of the people who currently test them. Both are considered below, followed by the negative impacts and what the design does about them.

Software quality, and why it is a safety question here. A defect in the graphical interface of a consumer application is usually an annoyance. In the class of application this project was evaluated against it is not. The target is an agricultural commerce platform used by smallholder farmers and buyers in Bangladesh, where a wrong price, an order routed to the wrong seller, or a total that does not match its line items is a direct financial loss to a party with very little margin to absorb it. Those are exactly the defects that a crash detector cannot see, for the reason set out in Section 2.1.4, and exactly the class that a specification-grounded oracle is built to catch. *Who is affected:* the end users of any application tested with this system. *How the design responds:* the oracle is derived from the requirements rather than from the absence of a crash, and every generated test cites the requirement it covers, so a defect report can be traced to the behaviour that was promised.

Access to systematic testing. Cost is not a neutral engineering detail in this context. A firm that cannot fund a dedicated quality-assurance team does not substitute a cheaper tool; it ships without systematic testing at all. Measured cost per executed test is therefore a societal variable as well as a budgetary one, and it is the reason NFR-D1 exists. *Who is affected:* small development firms, disproportionately in economies like Bangladesh's, and the users of the software they ship. *How the design responds:* unit cost is measured rather than assumed (DR-10), the expensive roles were moved to cheaper model tiers once the quality cost of doing so had been measured, and a new target is configured as a validated JSON profile through the dashboard rather than as a code change (DR-09), so adoption does not require an engineer who can modify the system.

Accessibility, as a by-product. The Android executor drives applications through the platform accessibility tree and the web executor through the accessibility-annotated DOM. The consequence is worth stating plainly: a control this agent cannot locate is, in general, a control that a screen-reader user cannot locate either. The agent's element-location failures are therefore an accessibility signal, and its failure log doubles as a rough accessibility audit of the application under test. *Who is affected:* users of assistive technology. *How the design responds:* controls are addressed by accessible role, name and content description rather than by pixel coordinates (Section 3.2), and each agent-attributed failure records the specific control that could not be reached. The agent does also receive a screenshot, which sharpens the signal rather than weakening it: a control the agent can see in the image but cannot name from the accessibility tree is exactly a control that is visible to a sighted user and invisible to a screen reader, and that divergence is the accessibility gap itself. This is a genuine benefit but an incidental one; it should not be presented as a designed accessibility audit, because the agent does not check contrast, focus order, or any other WCAG

criterion.

Language, and a failure that was silent. The project's text normalisation originally filtered input to the Latin range, which strips every non-Latin character. Two Bengali strings therefore both normalised to the empty string and were scored as nought per cent similar regardless of what they actually said. Bengali vowel signs are Unicode combining marks and are not covered by the usual word-character classes, so the bug survived casual inspection. It was fixed by normalising on Unicode category rather than on a Latin character class. The general point is larger than the bug: tooling built on Latin-script assumptions does not fail loudly on a Bengali interface, it fails silently and reports a plausible number. *Who is affected:* users and testers of software in Bengali and other non-Latin scripts. *How the design responds:* script-agnostic normalisation wherever user-visible text is compared.

Legal considerations. These are analysed in full in Section 3.2. In summary: dedicated disposable test accounts rather than production user data, no ingestion of user records into the knowledge graph, and destructive or communicative actions blocked at the configuration layer rather than discouraged in prose.

Negative impacts, and what is done about them

N1. Effect on quality-assurance employment. A tool whose stated purpose is to displace manual effort has a labour effect, and it would be dishonest to present it otherwise. The evidence available suggests augmentation rather than replacement so far: the closest field deployment in the literature reports a saving of 1.5 person-days per regression round on a single business line [7], which redirects effort rather than removing a role. The design choice that matters here is that the output is a natural-language test case a person can read, review and take ownership of, not an opaque verdict, so the human role shifts toward judgement rather than execution. At larger scale the effect on entry-level testing work is real and lies outside what this project can control.

N2. Automation bias. The most likely way this system causes harm is not by failing, but by being believed. An organisation that treats an automated verdict as authoritative will ship the defects the agent missed. *Response:* a reported defect is explicitly scoped as a candidate for human confirmation and never as a confirmed fault (Section 1.4); the generated report states its own degradations rather than hiding them (NFR-D2); and the attribution taxonomy makes it structurally impossible for the agent's own failure to be filed as an application defect (NFR-D3).

N3. Acting on a production system with live users. The evaluation target is in production.

An exploratory agent that decides for itself what to do is, by construction, capable of doing something irreversible. *Response:* the safety boundary is machine-enforced (DR-07) through blocked control texts and blocked URL patterns, out-of-scope requirements are withheld from the planner's citable list, and accounts used for testing are disposable.

N4. Prompt injection. The application's own on-screen text is fed to a model that also decides the next action, so content under an attacker's control could in principle direct the agent. *Response:* every prompt that carries observed application content labels it explicitly as untrusted data that must never be read as instructions, in line with the OWASP guidance for language-model applications [20].

N5. False reports wasting developer time. Each incorrectly reported defect costs investigation effort and erodes trust in the tool, which is the conflict recorded in Table 3.1. *Response:* the evidence-integrity requirements above. The honest limitation is that the false-positive rate has not been measured, because doing so requires the seeded-defect experiment described in Section 7.1.

6.4 Environmental Impact and Life Cycle Sustainability

Development

The development footprint is five developer workstations, one physical Android handset, emulator instances on those workstations, and a local graph database. No hardware was manufactured for the project; the handset is an existing personal device. The decision that matters most environmentally was taken for other reasons: model training and fine-tuning were placed out of scope (Section 1.4), and the project uses hosted inference through an API instead. Training is the dominant energy cost in most machine-learning work, so excluding it means this project's footprint is an inference footprint only.

Operation

The recurring environmental cost is hosted model inference, and it scales with the number of tokens processed rather than with wall-clock time. Three properties of the design reduce that volume. All three were adopted to control money rather than energy, but the mechanism is the same.

- Roughly nine tenths of the work in each execution happens on the device or in the browser rather than in a model, so most of a campaign consumes no inference at all.

- The executor and investigator roles run on smaller, cheaper model tiers, adopted once the quality cost of the downgrade had been measured.
- Context is budgeted rather than accumulated. Findings are atomic and deduplicated on write (DR-06), so a discovery made in round one is carried into round twenty as a single line with an observation count instead of being restated in full each time. Before that change, one test case's prose occupied the majority of the next generation prompt.

The literature supports token volume as the correct lever. AutoDroid halves its prompt length by merging functionally equivalent interface nodes and reports the query cost falling by the same proportion [5]; AppAgentX reduces tokens per task from 9.26k to 4.94k by replacing repeated reasoning with stored shortcuts [4]. Against these, the per-application figures for a two-hour exploratory campaign reported by DroidAgent [8] indicate the scale of inference that an unoptimised design consumes.

Device-side energy is smaller but not zero: a handset driven continuously for the length of a campaign, plus a graph database left running between campaigns. The honest limitation is that this project measured *monetary* cost, not energy. Token count is a reasonable proxy, but converting it to energy or carbon requires the provider's hardware and grid mix, which are not disclosed. No carbon figure is therefore claimed anywhere in this report.

Data life cycle, which is the real issue

The knowledge graph accumulates screenshots, accessibility dumps and execution traces of a production application, gathered through accounts that can sign in to it. That is sensitive local test evidence with a retention question attached, not neutral telemetry, and three concrete problems follow.

- E1. Decay is not deletion.** A knowledge half-life (default 90 days) downweights stale knowledge in retrieval so the agent adapts when an application changes. It keeps the graph *relevant*; it does nothing to bound how large it grows or how long the underlying evidence persists. *Mitigation:* a retention policy that actually deletes, applied to screenshots and raw traces separately from the derived findings that give them value.
- E2. Graph snapshots are large, and permanent if mishandled.** A full export of the knowledge graph runs to tens of megabytes and carries the same evidence the live graph does. Version control is the wrong home for such a snapshot, because anything committed to a repository is effectively permanent in its history. *Mitigation:* snapshots are held outside version control, under the same controls as the test credentials and with a stated retention period.
- E3. Screenshots are the bulk and the most sensitive part.** *Mitigation:* store them outside

the repository with an explicit lifetime, and keep the derived caption and structural signature, which is what retrieval actually uses, rather than the image, once that lifetime expires.

Retirement

The system is four processes, one database and no bespoke hardware, so decommissioning it is deleting a database, revoking API keys and disabling the test accounts. Nothing becomes electronic waste. The obligation that outlives the software is the data described above, and it is the part that a retirement plan has to name explicitly rather than assume will disappear with the code.

6.5 Ethics

6.5.1 Equity and Inclusivity

Three groups could be excluded by a system of this kind, and each was considered.

Users of software written in non-Latin scripts. The normalisation defect described in Section 6.3 is the concrete instance: a component that silently scored every pair of Bengali strings as completely dissimilar. It is representative of a wider pattern in which tooling developed against English interfaces degrades invisibly elsewhere, and the response was to normalise on Unicode category so that any script is handled on the same footing.

Users of assistive technology. Covered in Section 6.3: because the agent is confined to what the accessibility layer exposes, an application that is opaque to it is generally also opaque to a screen reader, and the agent's failures make that visible rather than hiding it.

Organisations without the budget for quality assurance. The measured unit cost and the configuration-driven target system are what put this within reach of a team that cannot fund a dedicated testing function.

The limitation to state honestly is that the agent is only as inclusive as the interface it is given. An application built with a toolkit that exposes no stable identifiers and no text through the accessibility tree, which is precisely constraint C4, is one the agent tests poorly. That is a real exclusion and it falls on exactly the applications that are already least accessible.

6.5.2 Accountability and Personal Responsibility

Responsibility for a system that issues correctness judgements has to be located deliberately, because the failure mode is a machine-authored claim that a human then acts on.

At the system level the position taken throughout this project is that **the agent is not accountable for its verdicts; the people operating it are**. Three mechanisms make that position operable rather than rhetorical. A reported defect is scoped as a candidate requiring human confirmation (Section 1.4). The three-way fault attribution taxonomy (NFR-D3) has exactly one definition in the source tree, so an agent limitation cannot be reported as an application defect through drift between components. And the cross-process degradation register (NFR-D2) records every fallback taken, on the principle that a system that hides its own degradation is worse than one that fails loudly.

Two decisions during the work illustrate the same principle applied to the team's own conduct. The team put its own codebase through a structured internal review against an explicit severity rubric, rather than assuming that code it had written was code it had checked, and scheduled the resulting work accordingly. And the evaluation reports autonomy and coverage without adjustment, including where those numbers are unflattering, because a testing tool that presents its own results selectively has disqualified itself from the job.

TODO (team): add the per-member statement this indicator also asks for, naming who owned which subsystem and who is answerable for it. That is the same information as Section 6.2.1 and should be consistent with it.

6.5.3 Proper Use of Intellectual Property

Prior work and literature. Every empirical claim taken from the research literature in Chapter 2 is attributed to the paper it came from, and the figures were read from those papers directly rather than from secondary summaries. Where a work was surveyed rather than read in full, this is stated at the point of use (Section 2.1.6) instead of being blurred.

Third-party software. The system is assembled from open-source components and hosted services. The principal Python dependencies are FastAPI and Uvicorn (HTTP services), Pydantic (schemas), the Neo4j driver, LangChain and LangGraph (agent orchestration), Playwright (browser execution), mobilerun (Android device execution), fastembed and NumPy (embeddings), markdown, pypdf and fpdf2 (document handling), structlog and psutil (operations), and an OpenRouter client for hosted inference. The dashboard uses React built with Vite. Neo4j itself is the graph database. Model inference is obtained commercially through OpenRouter and is governed by that provider's terms of service.

TODO (team): before submission, list each of the above with its licence (most are MIT, Apache-2.0 or BSD; Neo4j Community is GPLv3 and Playwright is Apache-2.0) and confirm that the project's own intended licence is compatible with all of them. pip-licenses will generate the table.

The partner's material. The technical requirements document ETA-TR-2026-001 is not public. It is cited for traceability, and requirement identifiers and titles are reproduced where they are needed to show what was delivered, but no confidential content beyond that appears in this report.

Generative AI. Generative AI tools were used during this project, and the Candidates' Declaration requires that use to be disclosed rather than concealed. Their role was assistance with implementation, refactoring, documentation and drafting, under the team's direction and review; the architecture, the requirements analysis, the experimental decisions and the interpretation of results are the team's own. All generated code was read, tested and, where wrong, corrected before it was kept.

TODO (team): replace the sentence above with the team's own precise account of how these tools were used, and check it against whatever disclosure wording the department requires. Do not submit a generic statement.

6.5.4 Professionalism and Ethical Codes

The ACM Code of Ethics and the IEEE Code of Ethics both place an obligation on the engineer to avoid harm, to be honest about the limitations of their work, and to accept responsibility for professional judgements. The IEB code makes the same demand of the engineer's duty to the public. Three decisions in this project were settled by those obligations rather than by convenience, and each is documented at the point where it was taken.

Honesty about what the work establishes. The review in Section 2.1.4 concludes that this project cannot report precision or recall for defect detection without seeding known faults, and says so, rather than substituting a coverage number that would look like a result. The corresponding ACM obligation is to be honest and trustworthy, including about the limits of one's own competence and evidence.

Avoiding harm to a third party's users. The evaluation target is a production system with real users. Registration, one-time-password verification, payment and account deletion were placed out of scope and blocked in configuration rather than merely discouraged (DR-07), on the straightforward reading that an engineer may not risk another party's users to make their own evaluation more complete.

Applying to our own system the standard we ask of others. The test account's secret is passed

only to the component that has to type it, and never to the planner, which receives the account role alone (NFR-D6). The planner would produce marginally better-grounded tests with the full credential in its context, so the decision cost a little capability to uphold a principle that would have been invisible had it gone the other way. An engineer's obligations do not lapse because the system under discussion is their own.

Chapter 7

Conclusion

7.1 Conclusion and Future Work

Achievement against the objectives

Section 1.4 stated eight objectives, each with the check that would settle it. Table 7.1 reports the outcome of each check. Chapter 5 carries the evidence; this section states the verdict and does not repeat the argument.

Table 7.1: Outcome against each stated objective

Objective	Outcome	Status
O1 Knowledge graph	A project-scoped property graph ingests a requirements specification, a design export and a defect history independently of one another, and is additionally grown from the agent's own observations of the running application. Each source was verified by ingesting it alone and querying the resulting nodes and relationships.	Met.
O2 Self-selected, non-duplicate, requirement-citing tests	No test objective enters the loop from outside. Proposals are rejected structurally and as duplicates, with an embedding-cosine check behind a lexical pre-filter. Requirement citation is enforced at both construction paths after a defect that allowed decorated identifiers to bypass it.	Met.

continued on the next page

Table 7.1: Outcome against each stated objective (*continued*)

Objective	Outcome	Status
O3 Autonomous execution on two platforms with three-way attribution	Test cases execute unattended against a physical Android device through the accessibility service and against a live web application through a browser, returning a verdict with an explicit application, agent or environment attribution.	Met in capability. <i>Insert the measured autonomy rate from Chapter 5 and state plainly whether the campaign target was reached.</i>
O4 Closed learning loop	Execution outcomes persist as deduplicated findings, navigation paths, error patterns and strategy scores, and are read back into the next round's prompt. The loop was verified by showing that a proposal changes when, and only when, the corresponding knowledge is present in the graph.	Met.
O5 The partner's eight requirement families	All eight families and both non-functional requirements are implemented, with the traceability matrix in Table 3.2. Two carry honest qualifications: defect intelligence (ETA-REQ-301) has been exercised on synthetic data only, and cross-dimensional transfer (ETA-REQ-304) is implemented but unvalidated.	Met, with two qualifications.
O6 Application-agnostic	Application-specific values arrive through configuration rather than code, enforced by a repository-wide generality audit, and the same unmodified system was run against more than one application on two different platforms.	Met.
O7 Measured cost within budget	Cost per executed test is reconciled against the provider's billing record rather than estimated, and the expensive roles were moved to cheaper model tiers after the quality cost of doing so had been measured.	Met. <i>Insert the final figure and the reconciliation from Chapter 5.</i>

continued on the next page

Table 7.1: Outcome against each stated objective (*continued*)

Objective	Outcome	Status
O8 Operator surface	A dashboard shows the live session, coverage, the growing application model and the planner trace; a run report exports as a PDF whose numbers are computed from the graph while the language model supplies prose only.	Met.

What the project establishes

Four claims are supported by the work.

First, an agent can select its own test objectives from a specification and a learned model of a running application, execute them without supervision, and do so at a unit cost low enough that the budget stops being the binding constraint on a campaign. This is the capability the industry partner asked for, and it is the part of the problem that the reviewed literature attempts least often: of the seven systems examined in Chapter 2, five are handed their objective and only one generates its own.

Second, the eight requirement families of ETA-TR-2026-001 are implementable as one coherent system rather than as eight separate features. Defect history, navigation memory, experiential learning, dimensional partitioning, self-healing, risk scoring, quality metrics and anomaly detection share a single graph, a single attribution vocabulary and a single planner, and each is reachable from the others. That integration, rather than any individual mechanism, is what makes the learning loop close.

Third, the architecture is not tied to one application or one interface technology. The same planner, the same graph and the same attribution taxonomy drove campaigns against two Android applications, including a production commerce platform with live users, and against four web targets ranging from a survey platform to a demonstration storefront to an encyclopaedia. Adding a target is a validated configuration file, not a code change.

Fourth, evidence integrity can be made a structural property rather than a matter of diligence. Silent fallbacks are forbidden and recorded across process boundaries; fault attribution has exactly one definition in the source tree, so an agent limitation cannot drift into being reported as an application defect; and a model-provider outage is classified as an environment fault rather than as a discovery. None of the seven reviewed systems distinguishes these three cases at all (gap G4), and the absence shows in their results.

What this kind of testing delivers

It is worth being explicit about what an exploratory agent is for, because the most convenient metric is not the most valuable output.

Defects nobody wrote a test for. The whole point of exploration is to reach behaviour that was never anticipated, which is precisely the behaviour no benchmark contains. That is an awkward property to measure and an excellent property to have, and it is why the field has fallen back on crash counts rather than abandoning the technique.

A map of where an interface is hard to use. An agent that reads the same accessibility layer as assistive technology, and that records every point at which it could not find a control, could not reach a screen, or circled a flow without progressing, produces something more useful than a pass rate: a list of the places where the application is genuinely difficult to navigate. A wizard that traps the agent is a wizard that will confuse a user, and a control the agent cannot see is one a screen reader cannot see either (Section 6.3). The three-way attribution taxonomy is what makes this readable, because it separates those observations from defect reports instead of mixing them together.

Durable artefacts that outlive the campaign. A run leaves behind a navigable model of the application's screens and transitions, readable test cases that each cite the requirement they cover, deduplicated findings carrying an independent-observation count, and a coverage report against the requirement set. Requirement coverage in particular is a number none of the reviewed systems can report at all, because none of them ingests a specification.

The open measurement, and why it is the field's rather than this project's

One figure is not yet available: the proportion of the defects actually present in a target that the agent found, and the proportion of its reports that were spurious. Section 2.1.4 sets out why. Computing precision and recall requires faults whose location is known in advance, and the two reviewed systems that report such figures obtained them by injecting faults by hand [7], because naturally occurring defects in a mature application are neither catalogued nor conveniently placed.

The useful observation is that specification grounding attacks the root of that problem rather than working around it. Where a crash-only oracle can measure nothing but crashes, an agent that knows what the application was supposed to do has, in principle, an oracle for every requirement it can cite. Turning that principle into a measured precision and recall is the experiment described in F1 below, and it is a contribution the field is missing rather than a gap peculiar to this implementation.

Boundaries of the evidence

Three boundaries should be read alongside the results in Chapter 5. The evidence base is a set of campaigns across the six targets named above rather than a standard benchmark suite, so comparisons with published figures are indicative rather than like-for-like. The system is scoped to local, developer-operated execution on a trusted host, so its behaviour in shared infrastructure is outside what was evaluated. And every result is conditioned on a budget, because exploratory testing has no natural stopping point (difficulty D2), which makes coverage a quantity that is reported at a given spend rather than one that is ever complete.

TODO (team): revisit this whole section once Chapter 5 is final. Three things need reconciling with the measured results: the two slots in Table 7.1 (autonomy rate for O3, cost figure for O7); the list of six targets above, which should name only those that actually carried a full campaign and say how many rounds each ran; and the opening claim about unit cost, which must match the reconciliation reported in Chapter 5. If the campaign against the target with documented defects returns usable data, add it here as direct evidence on the open measurement above and update F1 accordingly.

Future work

The items below are ordered by what they would add to the evidence, not by effort.

- F1. A defect-injection evaluation harness.** Section 2.1.4 shows that this field cannot report precision or recall because it has no labelled ground truth, and that the only two systems reporting such figures obtained them by injecting faults by hand [7]. A reusable harness that seeds a catalogue of functional faults into a build, runs an agent against seeded and clean variants, and scores the result would turn a one-off manual expedient into shared measurement infrastructure. It is the most valuable extension available, because it is the one that would let any agent of this kind state what fraction of real defects it finds.
- F2. A second oracle tier that inspects durable state.** The system currently derives its expectation from the specification and checks it against what the interface displays. A complementary tier would verify the effect instead of the appearance, by inspecting application or database state after a test completes, in the manner of state-checking benchmark environments [15, 16]. It would not cover every assertion, but where it applies it removes the agent from its own assessment and catches unintended side effects that no screen shows.
- F3. Synthesis of a deterministic regression suite.** An exploratory campaign produces exactly the raw material a scripted suite is written from: a confirmed behaviour, the shortest path that reaches it, and the assertion that distinguishes right from wrong. Promoting confirmed findings into generated, replayable test scripts would join the two halves

of Table 3.5 that this report treats as complementary, letting the agent discover and a deterministic suite defend.

- F4. Extension to further device profiles.** The dimensional model (ETA-REQ-304) is built to partition knowledge by profile, platform and application, and the partner's interest is in transfer across all three. Extending execution to watch, television and tablet profiles, where the input modality changes to a rotary control or a directional pad, is the natural next axis and would put the transfer mechanism under real load.
- F5. Cached and selective visual grounding.** The system already reads pixels as well as structure: the executor is given a screenshot alongside the accessibility tree at every step, the planner receives a screenshot of the state it is writing a test for, and a perceptual hash settles screen identity when the accessibility tree is too thin to do so. What is missing is not the capability but its economics. An image is sent with every step whether or not that step needed one, and nothing derived from it is kept, so the same screen is interpreted afresh on every visit. The extension is to make the input selective and the output durable: attach an image only when the structural signature is ambiguous, and persist what the model reads from it, a screen caption and names for the controls the accessibility tree leaves unnamed, onto the corresponding node in the application model. That converts a recurring per-step image cost into a one-off per-screen cost, and it promotes a visual interpretation into shared knowledge that later rounds and other roles can read instead of re-deriving.
- F6. On-device and smaller-model execution.** Every execution currently depends on a hosted model reachable over the network. Running the executor on a small local model would remove that dependency and with it the per-test charge, the network latency and the need to send interface content off the device at all, which matters for testing applications whose screens contain data that cannot leave the organisation. Comparable work has already placed a seven-billion-parameter model on the handset itself [5].
- F7. Parallel execution and continuous-integration triggering.** The architecture already separates planning from execution across processes, so a single planner could drive several devices at once, and a campaign could be triggered by a build rather than by an operator. This is what would move the system from a tool an engineer runs to a service a pipeline consumes.
- F8. A learned device-control policy.** The system adapts through graph memory, strategy scores and prompt construction, which keeps it usable with hosted models it does not control. Training a policy directly from interaction outcomes is the alternative, and the reported gains are large [17]. It is the natural successor to this architecture rather than a competitor to it.

7.2 Continuous Engagement and Self-directed Learning

None of the following was taught in coursework. Each was identified as a gap during the project and closed by the team.

Technologies learned independently. Graph data modelling in Neo4j, including Cypher, uniqueness constraints and vector indexes used alongside lexical retrieval. LangGraph for orchestrating a multi-step agent as an explicit state machine rather than as a chain of calls. The Android accessibility service and ADB as a control surface, and the companion on-device application needed to make text entry reliable. Playwright for browser automation, including the accessibility-annotated snapshot and file-chooser handling. Tool-calling and structured-output modes on hosted language models. Sentence embeddings for semantic deduplication. React with Vite for the operator dashboard, and $\text{\LaTeX}$ for this report.

Literature engaged with. Seven recent systems were read in full and are analysed in Chapter 2: four GUI testing systems [6–8, 10] and three task automation agents whose memory mechanisms transfer to the testing problem [3–5]. A wider set of benchmark environments and grounding models was surveyed alongside them. The foundational definition of exploratory testing was taken from its source [?] rather than from a secondary description.

Standards and practice consulted. ISO/IEC/IEEE 29119 for testing vocabulary and documentation [9], ISO/IEC 25010 for the quality model [18], ISO/IEC/IEEE 29148 for requirements traceability [19], and the OWASP guidance for language-model applications for prompt injection and credential handling [20]. None of these was part of the syllabus.

Engineering practices adopted during the project. Three working principles were arrived at during the project rather than brought to it, and each changed the system more than any single paper did.

Degradation must be observable. A component that falls back quietly produces a result that looks acceptable and measures something other than what it claims. The team therefore made degradation a first-class recorded event, propagated across process boundaries, on the principle that a system whose output is a correctness judgement may not be allowed to fail silently. This is the origin of NFR-D2.

A measurement is built upward, not divided downward. An aggregate divided by a count conceals every assumption that went into the aggregate. The project’s unit cost was therefore established by instrumenting a single execution end to end and reconciling upward against the provider’s

record, rather than by apportioning a total, and the same discipline is why coverage is reported against an enumerated requirement set rather than inferred.

A property that matters is machine-checked. Application-agnosticism is easy to intend and hard to verify by reading, because an author reviews the code they remember writing. The team wrote an automated audit that sweeps the whole source tree for application-specific vocabulary, on the general principle that a requirement worth stating is a requirement worth testing, including when the system under test is one's own.

Bibliography

- [1] Android Developers, “UI/Application exerciser monkey,” <https://developer.android.com/studio/test/other-testing-tools/monkey>, 2012.
- [2] *Technical Requirements Document ETA-TR-2026-001: Exploratory Testing Agent — Next-Generation Enhancement*, Samsung Research and Development Institute Bangladesh, 2026, version 1.0, 29 June 2026. Industry partner document, not publicly available.
- [3] C. Zhang, Z. Yang, J. Liu, Y. Han, X. Chen, Z. Huang, B. Fu, and G. Yu, “AppAgent: Multimodal agents as smartphone users,” 2023, arXiv:2312.13771 [cs.CV].
- [4] W. Jiang, Y. Zhuang, C. Song, X. Yang, J. T. Zhou, and C. Zhang, “AppAgentX: Evolving GUI agents as proficient smartphone users,” 2025, arXiv:2503.02268 [cs.AI].
- [5] H. Wen, Y. Li, G. Liu, S. Zhao, T. Yu, T. J.-J. Li, S. Jiang, Y. Liu, Y. Zhang, and Y. Liu, “AutoDroid: LLM-powered task automation in Android,” in *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*, Washington D.C., USA, 2024.
- [6] Z. Wang, W. Wang, Z. Li, L. Wang, C. Yi, X. Xu, L. Cao, H. Su, S. Chen, and J. Zhou, “XUAT-Copilot: Multi-agent collaborative system for automated user acceptance testing with large language model,” 2024, arXiv:2401.02705 [cs.AI].
- [7] Y. Hu, X. Wang, Y. Wang, Y. Zhang, S. Guo, C. Chen, X. Wang, and Y. Zhou, “AUITestAgent: Automatic requirements oriented GUI function testing,” 2024, arXiv:2407.09018 [cs.SE].
- [8] J. Yoon, R. Feldt, and S. Yoo, “Autonomous large language model agents enabling intent-driven mobile GUI testing,” 2023, arXiv:2311.08649 [cs.SE].
- [9] *ISO/IEC/IEEE 29119-1: Software and Systems Engineering — Software Testing — Part 1: General Concepts*, International Organization for Standardization, 2022.
- [10] Z. Liu, C. Chen, J. Wang, M. Chen, B. Wu, X. Che, D. Wang, and Q. Wang, “Make LLM a testing expert: Bringing human-like interaction to mobile GUI testing via functionality-aware decisions,” in *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, Lisbon, Portugal, 2024, arXiv:2310.15780.

- [11] Y. Li, Z. Yang, Y. Guo, and X. Chen, “DroidBot: A lightweight UI-guided test input generator for Android,” in *Proceedings of the 39th IEEE/ACM International Conference on Software Engineering Companion (ICSE-C)*, 2017.
- [12] ———, “Humanoid: A deep learning-based approach to automated black-box Android app testing,” in *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019, pp. 1070–1073.
- [13] R. Coppola and E. Alégroth, “A taxonomy of metrics for GUI-based testing research: A systematic literature review,” *Information and Software Technology*, p. 107062, 2022.
- [14] T. Su, J. Wang, and Z. Su, “Benchmarking automated GUI testing for Android against real-world bugs,” in *Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2021, pp. 119–130.
- [15] C. Rawles *et al.*, “AndroidWorld: A dynamic benchmarking environment for autonomous agents,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [16] H. Trivedi *et al.*, “AppWorld: A controllable world of apps and people for benchmarking interactive coding agents,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers*, 2024, pp. 16 022–16 076.
- [17] H. Bai *et al.*, “DigiRL: Training in-the-wild device-control agents with autonomous reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [18] *ISO/IEC 25010: Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model*, International Organization for Standardization, 2023.
- [19] *ISO/IEC/IEEE 29148: Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, International Organization for Standardization, 2018.
- [20] *OWASP Top 10 for Large Language Model Applications*, Open Worldwide Application Security Project, 2025, <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.